Contents

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System **4**
Document Map . . . 4
How to Read This Document . . . 5
The System in Brief . . . 5

Section 01 — Motivation, Problem Statement & System Overview **5**
1.1 The Problem I Was Trying to Solve . . . 5
1.2 What Existed, and Why It Was Insufficient . . . 6
1.3 What Study-Hub Is . . . 6
1.4 The Technology Decision at the Highest Level . . . 6
1.5 The Constraint That Shaped Everything: Free Infrastructure Only . . . 7
1.6 The Architecture in One Paragraph . . . 7
1.7 The Three-Layer Mental Model . . . 7

Section 02 — Technology Selection: Rationale & Rejected Alternatives **8**
2.1 The Question Behind Every Tool Choice . . . 8
2.2 Runtime: Node.js Over Python/Flask . . . 8
2.3 Web Framework: Express Over Alternatives . . . 9
2.4 Database: PostgreSQL Over MongoDB . . . 9
2.5 Authentication: JWT Over Sessions/Cookies . . . 10
2.6 AI Provider: OpenRouter Over OpenAI, Anthropic, or Local Models . . . 10
2.7 Frontend Framework: Vanilla JavaScript Over React/Vue/Svelte . . . 11
2.8 Hosting: Render.com Over Heroku, Fly.io, Railway, or AWS . . . 11

Section 03 — Frontend Language Choice: HTML as Structural Foundation **12**
3.1 Why HTML Deserves Its Own Section . . . 12
3.2 The Single HTML File Decision . . . 12
3.3 Semantic Markup Over Generic Divs . . . 13
3.4 The Hidden/Show Pattern for View Switching . . . 13
3.5 The Auth Screen as a Separate Layer . . . 13
3.6 Data Attributes as JavaScript Hooks . . . 14
3.7 The PDF Viewer Implementation . . . 14
3.8 What the HTML Architecture Enabled . . . 14

Section 04 — Styling Architecture & the Multi-Theme Decision **15**
4.1 The Problem With Just Writing CSS . . . 15
4.2 The File Architecture . . . 15
4.3 CSS Custom Properties as a Theme System . . . 16
4.4 The Glass Theme: A Different Problem . . . 16
4.5 The Custom Markdown Renderer . . . 17
4.6 Animation Philosophy . . . 18
4.7 The Theme Persistence Problem . . . 19

Section 05 — JavaScript: The Single-Language Stack Decision **19**
5.1 The Language Is Not the Architecture . . . 19
5.2 Async/Await Over Callbacks and Raw Promises . . . 20
5.3 The IIFE Module Pattern . . . 20
5.4 The `storage.js` Module: Lessons From Getting It Wrong . . . 21
5.5 The `api.js` Module: One HTTP Layer to Rule Them All . . . 22
5.6 The Event System for Cross-Module Communication . . . 23
5.7 The `helpers.js` Module: Pure Utilities . . . 23

Section 06 — Codebase Architecture: Separation of Concerns in Practice **24**
6.1 Why Structure Deserves Its Own Section . . . 24
6.2 The Top-Level Split: Backend and Frontend . . . 24
6.3 Backend Architecture: The Four-Layer Model . . . 25
6.4 The Decision to Have a `routes/index.js` File . . . 25
6.5 Frontend Architecture: Utils-First Dependency Order . . . 26
6.6 Environment Variables and the `.env` Pattern . . . 26
6.7 The `uploads/` Directory and the Ephemeral Disk Discovery . . . 27
6.8 The `scripts/` Directory: Operational Tooling . . . 27

Section 07 — Server Design: Express Middleware Pipeline & Security **28**
7.1 The Server as a Pipeline . . . 28
7.2 The Startup Sequence Problem . . . 28
7.3 Security Headers: The Helmet Configuration . . . 29
7.4 The CORS Configuration and the Bug It Caused . . . 30
7.5 Rate Limiting: Two Different Limiters for Two Different Threats . . . 30
7.6 The SPA Fallback Route . . . 31
7.7 Error Handler Middleware . . . 31
7.8 What I Would Add if Not For the Free Tier . . . 32

Section 08 — Data Modelling: Schema Design & PostgreSQL Selection **32**
8.1 The Data Model as the Foundation . . . 32
8.2 Why UUID Primary Keys Over Serial Integers . . . 33
8.3 The Core Schema: Seven Tables . . . 33
8.4 The Tags Column: PostgreSQL Arrays . . . 34
8.5 Full-Text Search: to_tsvector and to_tsquery . . . 34
8.6 The Dynamic Query Builder: A Design Problem I Fought With . . . 35
8.7 Connection Pooling and the Pool Configuration . . . 36

Section 09 — Authentication: bcrypt, JWT, and the Statelessness Decision **36**
9.1 The Authentication Problem . . . 36
9.2 Password Storage: bcrypt at Cost Factor 12 . . . 37
9.3 JWT: Why Stateless Tokens . . . 38
9.4 The Authentication Middleware . . . 38
9.5 JWT Expiry: The Headache I Hit in Production . . . 39
9.6 The Input Validation Layer . . . 40

Section 10 — REST API Design: Conventions, Ownership, and Consistency **41**
10.1 Why API Design Is a Separate Decision From Implementation . . . 41
10.2 REST as a Design Style, Not a Religion . . . 41
10.3 The Invariant Response Format . . . 42
10.4 HTTP Status Code Discipline . . . 42
10.5 Ownership Enforcement at the Query Level . . . 43
10.6 The Chat Endpoint: The Most Complex Route . . . 43
10.7 The PDF Streaming Endpoint's Auth Exception . . . 44
10.8 API Versioning: A Decision I Deferred . . . 44

Section 11 — SPA Architecture: Module Pattern Without a Build Tool **44**
11.1 The Build Tool Question . . . 45
11.2 The IIFE Module Pattern in Practice . . . 45
11.3 State Management: Plain JavaScript Objects . . . 46
11.4 DOM Querying: The `el()` Shorthand and Why I Cached References . . . 47
11.5 The Optimistic UI Pattern . . . 47
11.6 The Typing Indicator: UX Engineering . . . 48
11.7 The `setTimeout` Anti-Pattern I Deliberately Used . . . 49

Section 12 — Notes Subsystem: Full-Text Search & Dynamic SQL Construction **49**
12.1 The Notes Feature as the Core of the System . . . 49
12.2 The Folder-Note Hierarchy . . . 50
12.3 The Pinned Notes Ordering Problem . . . 50
12.4 The Full-Text Search Pipeline . . . 50
12.5 The Note Editor: Auto-Save vs. Explicit Save . . . 51
12.6 The Tags System: Comma-Separated Input to Array . . . 52
12.7 Soft Delete via Archiving . . . 52

Section 13 — PDF Subsystem: Upload Pipeline, Text Extraction & Storage **53**
13.1 Why PDFs Are Architecturally Different From Notes . . . 53
13.2 Multer: The Upload Middleware . . . 53
13.3 The PDF Parsing Problem . . . 54
13.4 The Upload Controller: Orchestrating the Pipeline . . . 55
13.5 In-Browser PDF Viewing: The iframe Solution . . . 56
13.6 The Ephemeral Disk Discovery in Context . . . 57

Section 14 — AI Chat: Context Injection, Model Fallback & Auto-Note Creation **57**
14.1 The Core Ambition and Its Constraint . . . 57
14.2 Why RAG Would Have Been Better . . . 58
14.3 The Context Injection Strategy . . . 58
14.4 The OpenRouter Integration and the Model Fallback Chain . . . 59
14.5 The System Prompt and the `[[CREATE_NOTE]]` Protocol . . . 60
14.6 Context Window Management: The Real Bottleneck . . . 61
14.7 Streaming Bugs: The Async Response Problem . . . 62
14.8 The Auto-Title Mechanism . . . 62

Section 15 — Holistic Architecture: Data Flow & Cross-Cutting Decisions **63**
15.1 Zooming Out: The System as a Whole . . . 63
15.2 The Complete Request Lifecycle . . . 63
15.3 Cross-Cutting Concern: Error Boundaries . . . 65
15.4 Cross-Cutting Concern: The Ownership Model . . . 65
15.5 The Module Communication Architecture . . . 65
15.6 Performance Characteristics Under Free-Tier Constraints . . . 66
15.7 Decisions Made for Simplicity That I Would Revisit . . . 66

Section 16 — Deployment: Free-Tier Infrastructure & the Ephemeral Disk **67**
16.1 The Gap Between Local Development and Production . . . 67
16.2 Infrastructure as Code: render.yaml . . . 67
16.3 What Changes in Production . . . 68
16.4 The Deployment Pipeline . . . 68
16.5 The Ephemeral Disk: Discovery, Diagnosis, and Response . . . 69
16.6 The 90-Day Database Expiry . . . 70
16.7 The Cold Start User Experience . . . 70
16.8 Environment Variables Managed Separately from Code . . . 70
16.9 What Production Deployment Taught Me . . . 71

Section 17 — Lessons Learned, Limitations & Future Work **71**
17.1 Overview . . . 71
17.2 What I Would Do the Same . . . 71
17.3 What I Would Do Differently . . . 72
17.4 Financial Constraint: What It Cost and What It Forced . . . 73
17.5 Known Limitations . . . 73
17.6 Future Work: Priority Order . . . 74
17.7 Closing Reflection . . . 74

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System

Author: Arnab Mukhar Biswas
Document Type: Research Documentation — First-Person Engineering Account
Status: Complete

Abstract

The proliferation of AI-powered productivity tools has created a widening gap between what is commercially available — expensive, opaque, and data-hungry — and what an independent developer can build under real-world constraints. This paper documents the design, implementation, and deployment of Study-Hub: a full-stack personal knowledge management system integrating note-taking, PDF ingestion, and conversational AI assistance. The system was built under strict financial constraints — exclusively free-tier infrastructure — which forced a series of deliberate architectural tradeoffs that shaped the final design in profound ways. I describe each decision chronologically: what I was trying to solve, what I considered, what I rejected, why I chose what I chose, and what broke along the way. The result is a working, deployed web application with a Node.js/Express backend, PostgreSQL database, vanilla JavaScript frontend, and a multi-model LLM fallback chain powered by OpenRouter's free tier. In lieu of Retrieval-Augmented Generation (RAG) — which would have required paid vector database infrastructure — I designed a context-injection and truncation strategy that approximates relevant retrieval at zero cost. I also document the construction of a custom markdown rendering engine, the resolution of streaming and CORS bugs, the JWT token lifecycle, and the ephemeral disk problem discovered post-deployment.

Document Map

File	Research Section
`00_INDEX.md`	Abstract & Document Map (this file)
`01_BIG_PICTURE.md`	Motivation, Problem Statement & System Overview
`02_TOOLS_SETUP.md`	Technology Selection: Rationale & Rejected Alternatives
`03_HTML_FOUNDATIONS.md`	Frontend Language Choice: HTML as Structural Foundation
`04_CSS_FOUNDATIONS.md`	Styling Architecture & the Multi-Theme Decision
`05_JAVASCRIPT_FOUNDATIONS.md`	JavaScript: The Single-Language Stack Decision
`06_PROJECT_STRUCTURE.md`	Codebase Architecture: Separation of Concerns in Practice
`07_BACKEND_NODE_EXPRESS.md`	Server Design: Express Middleware Pipeline & Security
`08_DATABASE_POSTGRESQL.md`	Data Modelling: Schema Design & PostgreSQL Selection
`09_AUTHENTICATION.md`	Authentication: bcrypt, JWT, and the Statelessness Decision
`10_API_DESIGN.md`	REST API Design: Conventions, Ownership, and Consistency
`11_FRONTEND_ARCHITECTURE.md`	SPA Architecture: Module Pattern Without a Build Tool
`12_NOTES_FEATURE.md`	Notes Subsystem: Full-Text Search & Dynamic SQL Construction
`13_PDF_FEATURE.md`	PDF Subsystem: Upload Pipeline, Text Extraction & Storage
`14_CHAT_AI_FEATURE.md`	AI Chat: Context Injection, Model Fallback & Auto-Note Creation
`15_SYSTEM_DESIGN.md`	Holistic Architecture: Data Flow & Cross-Cutting Decisions
`16_DEPLOYMENT.md`	Deployment: Free-Tier Infrastructure & Ephemeral Disk
`17_LESSONS_LEARNED.md`	Lessons Learned, Limitations & Future Work

How to Read This Document

This is not a tutorial. It is a first-person engineering account written for software engineers and researchers. Each section follows a chronological narrative: the problem I encountered, the decision space I mapped, what I rejected and why, and what I ultimately built. Code listings are included as concrete evidence of design decisions — not as instructions for replication.

The through-line of the entire paper is a single tension: **capability versus cost**. Every architectural choice eventually traces back to the constraint of operating entirely on free-tier infrastructure.

The System in Brief

Study-Hub is a three-feature web application: a Markdown note editor with folder and tagging support, a PDF upload and viewer with text extraction, and a conversational AI assistant that can read the user's notes and documents. All three features share a single authenticated user model. The stack is Node.js, Express, PostgreSQL, and vanilla JavaScript — no frontend framework, no build pipeline. The AI backend proxies to OpenRouter, which provides free access to open-source language models. Deployment is on Render.com's free tier.

The application is live. Every design decision documented here is already in production.

Section 01 — Motivation, Problem Statement & System Overview

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

1.1 The Problem I Was Trying to Solve

The modern student or researcher faces a fragmentation problem. Notes live in one application. PDFs live in another. An AI assistant — if available — lives in a third, knowing nothing about the first two. The workflow becomes: read a PDF, switch apps, ask the AI to explain something, manually paste in the relevant passage, receive an answer, switch back, write a note about it. Every step is friction. Every step is a context switch.

I wanted to eliminate that friction. I wanted a single environment where a user could write notes, read PDFs, and ask an AI questions about both — without ever leaving the application or manually pasting context. I wanted the AI to already know what the user was studying.

That is the problem Study-Hub was built to solve.

The secondary constraint — the one that shaped every architectural decision — was cost. I was building this independently, without funding, and I needed the entire infrastructure stack to operate for free. That ruled out most managed AI platforms (OpenAI's API is metered), most vector databases (Pinecone, Weaviate, and Qdrant all have metered tiers beyond small limits), and most cloud storage services. The constraint was severe but clarifying: it forced me to make every decision deliberately.

1.2 What Existed, and Why It Was Insufficient

Before building, I surveyed the existing landscape. The commercial tools — Notion AI, Obsidian Copilot plugins, ChatPDF — each solved part of the problem. Notion AI integrates AI with notes but treats PDFs as second-class citizens and costs money beyond a small free tier. Obsidian's AI plugins require API keys and configuration that is hostile to non-technical users. ChatPDF is excellent for single documents but has no persistent notes and no cross-document memory.

None of these gave me what I wanted: a unified, authenticated system where notes and PDFs and AI were first-class equals, running for free.

Building from scratch was not a choice made from ignorance of existing tools. It was a choice made after evaluating them and finding them all inadequate for this specific combination of requirements.

1.3 What Study-Hub Is

Study-Hub is a full-stack web application with three integrated features: a Markdown note editor, a PDF upload and in-browser viewer, and a conversational AI assistant that can read the user's notes and uploaded documents.

The architecture is a classical client-server model. The frontend is a Single-Page Application built in vanilla JavaScript — no framework, no build pipeline. The backend is a Node.js server using the Express framework. The database is PostgreSQL. The AI chat proxies to OpenRouter, which provides access to open-source language models on a free tier.

The system is fully multi-user: each user has an isolated account, and data ownership is enforced at the database query level. Authentication uses JSON Web Tokens (JWT). Passwords are hashed with bcrypt at a cost factor of 12.

1.4 The Technology Decision at the Highest Level

The most consequential early decision was the language choice. I chose JavaScript for both the frontend and the backend (via Node.js). This was not the inevitable choice — Python with Flask or Django would have been equally viable for the backend, and many developers would choose it for backend web work.

I chose JavaScript for one specific reason: cognitive overhead. Switching mental context between two languages in a single codebase has a cost. With a JavaScript-everywhere stack, I could move between frontend and backend code without switching language paradigms. The same `async/await` patterns, the same object literals, the same module conventions. That consistency was worth more to me than any marginal technical advantage either language had.

I explicitly rejected TypeScript at this stage. TypeScript's type system offers real benefits on large teams and long-lived codebases, but for a project of this scale — one developer, weeks not years — the compilation step and the ceremony of type annotations would slow down iteration without providing enough benefit to justify it. I made this tradeoff consciously and I stand by it.

1.5 The Constraint That Shaped Everything: Free Infrastructure Only

This deserves its own section because it affected nearly every subsequent decision.

Operating entirely on free-tier infrastructure meant:

No paid AI API. OpenAI's GPT-4 costs money per token. Claude's API costs money per token. I could not use either at scale without eventually incurring charges. This pushed me to OpenRouter, which aggregates access to open-source models (Llama, Gemma) and offers a genuinely free tier with rate limits but no credit card charges.

No vector database. Retrieval-Augmented Generation — the architecturally superior approach to injecting relevant document context into AI conversations — requires storing document embeddings in a vector database and performing semantic search at query time. All of the production-grade vector database services have metered pricing beyond their free tiers. I could not implement true RAG. Instead, I designed a context-injection strategy that truncates and prepends document content directly into the system prompt — a cruder but zero-cost approximation. I discuss this at length in Section 14.

Free-tier Render.com hosting. Render's free tier provided both the web service and a PostgreSQL database at no cost. The tradeoffs were significant: the service sleeps after 15 minutes of inactivity (cold start on the first request), the database expires after 90 days without upgrade, and — most painfully — the disk is ephemeral. Uploaded PDFs are deleted on every redeploy. I discuss the ephemeral disk problem and my response to it in Section 16.

Weak AI models. The free models available through OpenRouter are capable but not state-of-the-art. Llama 3.3 70B and Gemma 3 12B are genuinely useful study assistants, but they do not match the contextual precision of GPT-4 or Claude Opus. The AI feature of Study-Hub works well within these constraints, but I am honest about where the ceiling is.

These constraints were not failures. They were the conditions under which the project was built, and documenting them is part of documenting the project honestly.

1.6 The Architecture in One Paragraph

A user's browser downloads a single HTML file from the server. That HTML file loads a series of JavaScript modules which together constitute the application. When the user interacts with the application — writing a note, uploading a PDF, sending a chat message — the JavaScript sends HTTP requests to the Node.js/Express backend API. The API validates the request, performs database queries against PostgreSQL, and returns JSON responses. For AI chat requests, the backend additionally calls the OpenRouter API, builds a context-enriched message array, and returns the AI's response. Files uploaded by the user are stored on disk in a folder organized by user ID, with UUID filenames to prevent collisions and obscure original names. The database stores file metadata (name, size, path, extracted text) rather than file contents.

This is the classical three-tier web architecture. What makes Study-Hub interesting is not the architecture itself — it is the specific decisions made within each tier, the constraints that forced those decisions, and the bugs that emerged when theory met reality.

1.7 The Three-Layer Mental Model

Throughout this paper, I refer to three layers:

The frontend — everything that runs in the user's browser. HTML for structure, CSS for appearance, JavaScript for behaviour. The user sees and touches this layer.

The backend — the Node.js server running on a remote computer. It handles business logic, enforces authorization, talks to the database, and proxies AI calls. The user never sees this directly.

The database — PostgreSQL, running as a managed service on Render.com. The permanent store for all user data: accounts, notes, PDF metadata, chat history.

These three layers communicate: the frontend talks to the backend via HTTP; the backend talks to the database via a TCP connection on port 5432; the backend talks to OpenRouter via HTTPS. Understanding these communication paths is the foundation for understanding every decision that follows.

Section 02 — Technology Selection: Rationale & Rejected Alternatives

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

2.1 The Question Behind Every Tool Choice

For every technology in this stack, the question was the same: does this tool earn its place? Adding a dependency is a cost — it is something to install, understand, update, debug, and eventually migrate away from. A tool that solves a real problem at low complexity cost is worth it. A tool that solves a problem I do not have yet is not.

This section documents not just what I chose, but what I chose instead of — and why the rejected alternatives lost.

2.2 Runtime: Node.js Over Python/Flask

The first fork in the road was the backend runtime. The two realistic options for a solo developer building a JSON API were Node.js (JavaScript) and Python with Flask or FastAPI.

Python had a real argument: it is the dominant language in the AI/ML ecosystem. The libraries for vector embeddings, PDF processing, and LLM integration are richer in Python than in Node.js. If I had been building a system with deep ML components — training models, running local embeddings, working with NumPy arrays — Python would have won.

But Study-Hub's AI integration is entirely API-based. I am not running inference locally. I am constructing an HTTP request to OpenRouter and parsing the response. For that, the language difference is irrelevant.

What mattered was the frontend: the frontend was always going to be JavaScript. Running Node.js on the backend meant one language across the entire codebase. I could reuse the same mental model for `async/await`, the same understanding of closures and callbacks, the same module patterns. When I was debugging a problem that crossed the frontend/backend boundary — and there were several — not having to mentally switch language paradigms made the work faster.

I chose Node.js. I never regretted it.

2.3 Web Framework: Express Over Alternatives

Given Node.js, the framework choice was Express vs. Fastify vs. no framework (raw `http` module) vs. Hono or Koa.

Raw `http` module: The Node.js built-in HTTP module works, but building routing by hand — checking `req.url` and `req.method` in nested conditionals — produces code that is verbose, brittle, and hard to read. For a production application with 20+ routes, this was not viable.

Fastify: Fastify is faster than Express in benchmarks and has excellent TypeScript support. But its ecosystem is smaller, its documentation thinner, and its patterns less familiar. The performance advantage was irrelevant at the scale I was operating at — a free-tier Render service serving occasional requests does not need microsecond routing optimization.

Express: Express is the most widely used Node.js framework by a large margin. Its middleware model is conceptually simple (`(req, res, next) => {}`), its ecosystem is vast, and every problem I encountered had been encountered and documented by others. The `npm` ecosystem for Express integrations — `cors`, `helmet`, `express-rate-limit`, `multer` — is mature and well-maintained.

I chose Express. The decision was essentially about ecosystem maturity and documentation density. At this stage of development, the ability to find answers quickly was worth more than theoretical performance gains.

2.4 Database: PostgreSQL Over MongoDB

This was the decision I spent the most time on before building, and it is the one I am most confident about in retrospect.

MongoDB's argument: MongoDB is document-oriented and schemaless. Notes are essentially documents. The argument was that a note's flexible structure — optional tags, optional folder, optional color — maps naturally to a JSON document store. No migrations required when adding fields. Fast iteration.

Why I rejected MongoDB: The schemaless argument is seductive but misleading for a project like this. Study-Hub's data is deeply relational. A note belongs to a user, optionally belongs to a folder. A PDF belongs to a user, optionally belongs to a folder. A chat message belongs to a session, belongs to a user. A folder has a count of notes. These relationships are not incidental — they are the core of the data model. Querying them in MongoDB requires either embedding (which leads to data duplication and update anomalies) or `$lookup` aggregations (which are verbose and slower than SQL JOINs).

In PostgreSQL, the query `SELECT folders, COUNT(notes) FROM... LEFT JOIN... GROUP BY...` is natural, fast, and readable. The equivalent MongoDB aggregation pipeline is longer and harder to maintain.

Additionally, PostgreSQL's full-text search (`to_tsvector` / `to_tsquery`) gave me a working notes search feature for free, without any additional infrastructure. MongoDB's text search is capable but requires index configuration and, at scale, Atlas Search (which is not free).

I chose PostgreSQL. It was the right call. Every time I needed to add a filtered, sorted, joined query, the SQL was cleaner than any MongoDB equivalent I could construct.

2.5 Authentication: JWT Over Sessions/Cookies

The authentication architecture was a choice between **server-side sessions** (traditional cookies) and **stateless JWT tokens**.

Sessions: With session-based auth, the server generates a session ID on login, stores the session in a database or Redis cache, and sends the session ID as a cookie. On each request, the server looks up the session ID to verify the user. This is the older, battle-tested pattern.

JWT: With JWTs, the server signs a token containing the user's ID on login and sends it to the client. The client stores it (in `localStorage` or as a cookie) and sends it with every request. The server verifies the token's signature without any database lookup.

The reason I chose JWT was the statelessness. If I had used sessions, I would have needed a persistent session store — either a database table or Redis. Given that the free Render PostgreSQL expires after 90 days and I wanted to avoid any additional services, managing a session store added operational complexity.

JWTs are self-contained. The server can verify a token's validity purely cryptographically, with no I/O. This simplified the server architecture considerably.

The tradeoff I accepted: JWTs cannot be invalidated before they expire (without a token blacklist, which reintroduces state). A user who logs out still has a technically valid token until expiry. I set a short expiry window (24 hours) and accepted this limitation for a project of this scope.

2.6 AI Provider: OpenRouter Over OpenAI, Anthropic, or Local Models

The AI provider decision was dictated primarily by the free-infrastructure constraint.

OpenAI directly: GPT-4 and GPT-3.5-Turbo are excellent models. But OpenAI's API is metered — there is no genuinely free tier beyond an initial credit. At any real usage volume, costs accrue. Ruled out.

Anthropic directly: Claude is arguably the best model for document understanding and nuanced reasoning — my exact use case. But same problem: metered API, no sustainable free tier. Ruled out.

Local models (Ollama/LMStudio): Running models locally would eliminate API costs entirely. But a Render.com free-tier server does not have the hardware (GPU or sufficient RAM) to run a 7B+ parameter model. Local inference requires the user to run their own model, which is a hostile setup experience for most users. Ruled out.

OpenRouter: OpenRouter aggregates access to open-source models (Llama 3.3 70B, Gemma 3 12B, Mistral, and others) with a genuine free tier — rate-limited but functional. The interface is OpenAI-compatible, meaning the same JSON message format works without modification. I could also implement a model fallback chain: if the first model is rate-limited or unavailable, try the second, then the third. This resilience was something I could not easily achieve with a single-provider API.

OpenRouter was the only viable option given the constraints. The free models are weaker than GPT-4 or Claude Opus, and I document that limitation honestly. But they are capable enough for a study assistant — summarizing notes, answering questions about PDFs, generating structured notes on request.

2.7 Frontend Framework: Vanilla JavaScript Over React/Vue/Svelte

This was the decision that surprised people most when they reviewed the code. React is the dominant choice for interactive web applications. Why did I not use it?

React's argument: Component model, virtual DOM, ecosystem, JSX, state management. For a team-built application that will be maintained long-term, React's patterns are worth the setup overhead.

Why I rejected React: React requires a build pipeline — Vite, Webpack, or similar. A build pipeline means additional configuration, additional failure modes, longer feedback loops during development, and a compiled output that is harder to inspect directly. For a solo project with no team, this overhead did not pay off.

More importantly: Study-Hub's frontend has three main views (Notes, PDFs, Chat) and a set of modals. It is not a component-heavy UI with hundreds of instances of the same element. The complexity level that justifies a component framework was simply not present.

Vanilla JavaScript with a module pattern (IIFEs exporting global objects) gave me fine-grained control, zero build overhead, and outputs I could inspect directly in the browser's developer tools. Load the HTML file, see the running application — no transpilation step, no source maps required.

The tradeoff I accepted: no compile-time type checking, no tree-shaking, no hot module replacement. These costs are real and would matter on a larger project. For Study-Hub, they did not.

2.8 Hosting: Render.com Over Heroku, Fly.io, Railway, or AWS

Heroku: Used to be the standard free-tier platform. They eliminated the free tier in 2022. Ruled out.

Fly.io: Excellent platform, Docker-based, generous free tier. But deploying a Docker container adds a layer of abstraction and operational knowledge overhead. Render's `render.yaml` approach — just declare your service type and build command — was simpler to configure.

Railway: Similar free tier, similar ease of use. Railway's free tier was unstable during the period I was building, with pricing changes that made it unclear what would actually cost money. Render's free tier structure was better documented and more predictable.

AWS/GCP/Azure: Powerful, complex, and expensive without careful configuration. Using these for a student project risks accidentally incurring significant charges. The risk profile was wrong.

Render.com: Clear free tier with explicit limits (service sleeps after 15 minutes of inactivity; PostgreSQL expires after 90 days; disk is ephemeral). I knew exactly what I was getting. The `render.yaml` Infrastructure-as-Code approach meant my deployment config was committed to the repository — reproducible and version-controlled.

I chose Render and then discovered the ephemeral disk problem post-deployment. I document that discovery and its implications in Section 16.

Section 03 — Frontend Language Choice: HTML as Structural Foundation

---

3.1 Why HTML Deserves Its Own Section

In most web development accounts, HTML is treated as a given — of course you use HTML, it is the language of the web. But the decisions embedded in how HTML is used — specifically, the decision to write a Single-Page Application in a *single HTML file*, and the decision to use semantic markup rather than div-soup — had downstream consequences on every other frontend decision.

This section documents the HTML architecture of Study-Hub: why it is structured the way it is, what tradeoffs that structure creates, and the specific patterns I chose over alternatives.

3.2 The Single HTML File Decision

Study-Hub's entire frontend is `frontend/index.html`. One file. It contains the authentication screen, the Notes view, the PDFs view, the Chat view, all modals, and all structural markup for every part of the application.

This was not the only option. A multi-page approach would have created separate files: `login.html`, `notes.html`, `pdfs.html`, `chat.html`. The server would route to the appropriate file based on the URL. Each page would load only the CSS and JavaScript relevant to it.

I rejected multi-page in favour of a Single-Page Application for three reasons:

Persistent state. In a multi-page app, navigating from Notes to Chat causes a full page reload. The user's sidebar state, their active session, their theme preference — all of this must be re-fetched and re-applied on every navigation. In a SPA, state lives in JavaScript memory and persists across view switches. When I switch from Notes to Chat, the Notes module's state is simply hidden, not destroyed.

Transition quality. Full page reloads cause a visible flash. JavaScript-driven view switching can be instant and smooth. For an application where users navigate frequently between notes and the chat assistant, this matters perceptually.

Simpler server routing. In a multi-page app, the server needs routes for each page. If a user deep-links or refreshes on `/chat`, the server must know to return `chat.html`. In a SPA, the server always returns `index.html` and the JavaScript handles routing. The Express `app.get('*', ...)` catch-all fallback in `server.js` handles this with three lines of code.

The cost of a single HTML file is a larger initial download — the user downloads markup for all three views on first load, even views they may never visit. For Study-Hub, this cost is negligible: the HTML file is small (under 50KB), and all three views are core functionality that almost every user will visit.

3.3 Semantic Markup Over Generic Divs

Early in building the HTML, I made a decision to use semantic HTML elements wherever appropriate — `<nav>`, `<main>`, `<section>`, `<aside>`, `<header>`, `<article>`, `<button>` — rather than styling `<div>` elements to look like those things.

The motivation was not aesthetic. Semantic markup carries meaning that the browser, screen readers, and developer tools can interpret. A `<button>` element is keyboard-focusable by default, fires click events on Enter, communicates its role to screen readers, and has sensible default styling to override. A `<div>` styled to look like a button has none of these behaviours without explicit ARIA attributes and JavaScript event handling.

More practically: semantic markup made the CSS architecture cleaner. When I wrote `button.primary-action { ... }`, I was styling a meaningful element, not a class applied to an undifferentiated container.

3.4 The Hidden/Show Pattern for View Switching

Because all views coexist in a single HTML file, they cannot all be visible simultaneously. I used CSS `display: none` toggling to show and hide views:

```
<div id="notes-view" class="app-view hidden">...</div>
<div id="pdfs-view" class="app-view hidden">...</div>
<div id="chat-view" class="app-view hidden">...</div>
```

When the user navigates to Notes, `app.js` removes `hidden` from `notes-view` and adds it to all others. The transition is instantaneous from the user's perspective.

This pattern has a subtle implication: all three views' DOM nodes exist in memory simultaneously, even when not visible. JavaScript event listeners attached to hidden elements remain active. This required careful attention to ensuring that event handlers in one module did not accidentally respond to events from another module's elements. I solved this by keeping each module's event listeners scoped to that module's root element — a `data-view="notes"` boundary that all selectors stayed within.

3.5 The Auth Screen as a Separate Layer

The authentication screen — login and registration forms — sits outside the main application views in the HTML hierarchy:

```
<div id="auth-screen">...</div>
<div id="app-container" class="hidden">...</div>
```

On page load, the application is in one of two states: authenticated (show `app-container`, hide `auth-screen`) or unauthenticated (show `auth-screen`, hide `app-container`). This is determined by the presence of a valid JWT in `localStorage`.

I chose this structure over routing-based auth (where `/login` is a different route from `/app`) because it simplified the server: the server does not need to know whether a user is authenticated to serve the HTML file. Authentication state is entirely a client-side concern, determined locally from `localStorage`. The server's job is to reject unauthenticated API requests — which it does at the middleware level regardless of how the frontend is structured.

3.6 Data Attributes as JavaScript Hooks

A pattern I adopted consistently: JavaScript selects elements using `data-*` attributes rather than CSS classes whenever the selection is for behaviour rather than appearance.

```
<button class="note-card-pin-btn" data-note-id="abc-123" data-action="pin"> </button>
```

The CSS class (`note-card-pin-btn`) controls how the button looks. The data attributes (`data-note-id`, `data-action`) are what JavaScript reads to know what to do when the button is clicked.

This separation means I could freely refactor CSS class names without breaking JavaScript behaviour, and vice versa. Mixing the two — using CSS classes as JavaScript selectors — creates a coupling where renaming a class for aesthetic reasons breaks functionality. I wanted to avoid that.

3.7 The PDF Viewer Implementation

The in-browser PDF viewer was one of the more technically interesting HTML decisions. I needed to display PDFs inline — not trigger a download, not open a new tab — within the application's layout.

The solution was an `<iframe>` element:

```
<iframe id="pdf-iframe" src="" title="PDF Viewer"></iframe>
```

When the user opens a PDF, the `src` attribute is set to a streaming endpoint on the server:

```
/api/pdfs/{id}/stream?token=eyJ...
```

The browser's built-in PDF renderer handles the rest. This approach required no third-party PDF rendering library on the frontend.

The complication: the standard approach to accessing protected resources is sending the JWT in an `Authorization` header. But `<iframe>` elements cannot send custom headers — they are HTTP GETs controlled by the browser, not by JavaScript `fetch()`. I had to pass the authentication token as a URL query parameter instead, which the server then reads and validates. This is a small security tradeoff (tokens in URLs can appear in server logs and browser history) that I accepted for simplicity; the alternative — a pre-signed download URL system with short-lived tokens — was significantly more complex and not warranted at this scale.

3.8 What the HTML Architecture Enabled

The HTML decisions collectively enabled the rest of the frontend to be simpler. Because JavaScript had a stable, predictable DOM structure to work with — clear element IDs, data attributes for behaviour hooks, semantic elements with understood browser behaviour — the JavaScript modules could focus on logic rather than DOM wrangling.

The cost was a larger, more complex single HTML file. A developer opening `index.html` for the first time sees several hundred lines of markup representing three distinct views. Navigating that file requires understanding the overall structure. I mitigated this with consistent comments delimiting each view's section, but it remains a real drawback compared to a multi-file component architecture.

For the scale of this project, that tradeoff was correct. For a larger application with more views, I would revisit it.

Section 04 — Styling Architecture & the Multi-Theme Decision

4.1 The Problem With Just Writing CSS

The naive approach to CSS is to write it as you go — add a style when you need it, repeat values when you need them, fix conflicts when they appear. This works for small projects. It degrades badly at medium scale: colour values get inconsistent (`#1a1a2e` in one file, `#1a1a2f` in another), spacing is arbitrary, changing a theme colour requires a find-and-replace across dozens of files.

I had an additional constraint: Study-Hub needed to support four visual themes — Minimal Dark, Minimal Light, Glass Dark, Glass Light. Four themes across a full-featured application. Without an architectural approach to CSS, implementing four themes would mean maintaining four parallel copies of nearly every style rule. That was untenable.

The solution was a CSS custom properties (variables) system, combined with a deliberate file split.

4.2 The File Architecture

Rather than a single `styles.css` file, I split CSS across ten files with distinct responsibilities:

- `variables.css` — All theme tokens (colours, spacing, font sizes, border radii, shadow values)
- `reset.css` — Browser default style normalization
- `base.css` — Base typography and element-level defaults
- `components.css` — Reusable UI components: buttons, modals, badges, form elements
- `layout.css` — Page structure: navigation bar, sidebar, main content area
- `notes.css` — Notes-specific styles: cards, editor, folder list
- `pdfs.css` — PDF library and viewer styles
- `chat.css` — Chat interface, message bubbles, context panel
- `auth.css` — Login and registration screens
- `animations.css` — All `@keyframes` and transition definitions
- `glassmorphic.css` — Glass theme overrides applied on top of the base dark theme

Each file has a single responsibility. When I needed to change how note cards look, I opened `notes.css`. When I needed to adjust the sidebar layout, I opened `layout.css`. I never had to grep across files to find where a style lived.

The load order in `index.html` mirrors the dependency chain: `variables.css` first (everything else depends on the tokens), then `reset.css`, then `base.css`, then components, then layouts, then feature-specific files.

4.3 CSS Custom Properties as a Theme System

The core of the theme architecture is `variables.css`, which defines all design tokens as CSS custom properties on the `:root` selector:

```
:root {
  --bg-primary:     #0a0a0f;
  --bg-secondary:   #111118;
  --bg-tertiary:    #1a1a2e;
  --surface:        #1e1e2e;
  --border:         #2a2a3e;
  --text-primary:   #e2e8f0;
  --text-secondary: #94a3b8;
  --text-muted:     #64748b;
  --accent:         #43b97f;
  --accent-hover:   #369966;
  --radius-sm:      6px;
  --radius-md:      10px;
  --shadow-md:      0 4px 16px rgba(0,0,0,0.4);
}
```

These are the Minimal Dark defaults. Every component in every other CSS file uses these variables, never raw colour values:

```
/* components.css */
.primary-btn {
  background: var(--accent);
  color: var(--bg-primary);
  border-radius: var(--radius-md);
}
```

Switching to the Light theme is then purely a matter of overriding the variables on a parent element:

```
[data-theme="light"] {
  --bg-primary:   #f8fafc;
  --bg-secondary: #f1f5f9;
  --text-primary: #1e293b;
  /* ...all tokens redefined... */
}
```

When `data-theme="light"` is set on the `<body>` element, CSS cascade causes all child elements to inherit the new variable values. Every `var(--bg-primary)` reference in every component automatically resolves to the light theme value. The component styles themselves are unchanged.

This is a significant architectural win: implementing the Light theme required zero changes to `components.css`, `layout.css`, or any feature-specific file. Only `variables.css` needed new rules.

4.4 The Glass Theme: A Different Problem

The Glass Dark and Glass Light themes required a different approach. They are not just colour changes — they change the visual treatment of surfaces. Instead of solid backgrounds, surfaces become semi-transparent with a blur effect (glassmorphism):

```
[data-theme="glass-dark"] .card,
[data-theme="glass-dark"] .modal,
```

```
[data-theme="glass-dark"] .sidebar {
  background: rgba(255, 255, 255, 0.05);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid rgba(255, 255, 255, 0.1);
}
```

`backdrop-filter: blur(12px)` applies a Gaussian blur to whatever is behind the element. The `-webkit-` prefix is required for Safari compatibility. The `rgba()` background with low alpha creates the translucent surface effect.

This could not be handled by variable overrides alone — it required property changes (adding `backdrop-filter`) that do not exist in the default themes. So I created `glassmorphic.css` as a layer of structural overrides that applies only when the glass themes are active. The file is loaded after all other CSS so its rules win specificity conflicts.

The performance tradeoff of `backdrop-filter` is real: it forces GPU compositing of the affected elements and can cause frame rate drops on low-end hardware. I accepted this tradeoff for the aesthetic, but documented it as a known limitation.

4.5 The Custom Markdown Renderer

One of the more unexpected engineering challenges in the CSS section was rendering Markdown. The AI chat feature returns responses formatted in Markdown — headings, bullet lists, code blocks, bold text. I needed to render these as formatted HTML in the chat interface.

I did not want to import a third-party Markdown library for two reasons: bundle weight and customization. I needed the rendered HTML to use Study-Hub's CSS variables for colours and spacing, not the library's built-in styles. Overriding library styles is a fighting match I wanted to avoid.

So I wrote my own Markdown renderer. It handles:

- `# Heading 1`, `## Heading 2`, `### Heading 3`
- `**bold**` and `*italic*`
- `` `inline code` `` and ```` ```code blocks``` ```
- `- unordered list items`
- `1. ordered list items`
- `[links](url)`

The implementation is a series of regular expression substitutions applied in sequence:

```
const renderMarkdown = (text) => {
  if (!text) return '';
  let html = escHtml(text);                    // 1. Escape raw HTML first (XSS)

  // Code blocks (must come before inline code)
  html = html.replace(/```([\s\S]*?)```/g,
    (_, code) => `<pre><code>${code.trim()}</code></pre>`);

  // Headings
  html = html.replace(/^### (.+)$/gm, '<h3>$1</h3>');
  html = html.replace(/^## (.+)$/gm,  '<h2>$1</h2>');
  html = html.replace(/^# (.+)$/gm,   '<h1>$1</h1>');

  // Bold and italic
```

```
  html = html.replace(/\*\*(.+?)\*\*/g, '<strong>$1</strong>');
  html = html.replace(/\*(.+?)\*/g,     '<em>$1</em>');

  // Inline code
  html = html.replace(/`(.+?)`/g, '<code>$1</code>');

  // Lists
  html = html.replace(/^- (.+)$/gm,    '<li>$1</li>');
  html = html.replace(/(<li>.*<\/li>\n?)+/gs, '<ul>$&</ul>');

  // Paragraphs (double newlines become paragraph breaks)
  html = html.replace(/\n\n/g, '</p><p>');
  html = `<p>${html}</p>`;

  return html;
};
```

The critical line is the first one: `escHtml(text)` escapes raw HTML characters (`<`, `>`, `&`, `"`) before any Markdown processing. This prevents XSS (Cross-Site Scripting): if the AI ever returned something like `<script>alert('xss')</script>`, the escape converts it to `<script>...` and it renders as harmless text rather than executing.

The ordering matters throughout. Code blocks must be processed before inline code because the triple-backtick pattern must be captured before the single-backtick pattern runs. Headings must be processed before paragraph wrapping. Each substitution step must be mindful of what previous steps have already transformed.

This renderer is not a full CommonMark implementation — edge cases exist. But it handles everything the AI actually produces in practice, which is the only standard that matters for this use case.

4.6 Animation Philosophy

I used CSS animations (`@keyframes`) rather than JavaScript-driven animations for three reasons: performance (CSS animations run on the GPU compositor thread without blocking the JavaScript thread), simplicity (no `requestAnimationFrame` loops to manage), and declarativeness (animation is described in CSS where it belongs, not scattered through JavaScript).

The shimmer loading animation on the chat typing indicator is the most visible example:

```
@keyframes shimmer {
  0%   { background-position: -200% 0; }
  100% { background-position:  200% 0; }
}

.shimmer-line {
  background: linear-gradient(
    90deg,
    var(--bg-tertiary) 25%,
    var(--border)      50%,
    var(--bg-tertiary) 75%
  );
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
}
```

The shimmer effect moves a gradient horizontally across the element, creating the impression of light passing across a surface. Three lines of different widths together simulate a loading text block. The effect uses CSS variables for colours, so it automatically adapts to all four themes without any modification.

4.7 The Theme Persistence Problem

Storing and restoring the user's theme preference required coordination between CSS and JavaScript. The theme is persisted in `localStorage` and applied on page load. But applying it on page load creates a problem: if the HTML is rendered with the default dark theme and then JavaScript reads `localStorage` and switches to the light theme, the user sees a flash of dark content before the switch.

I solved this with an inline `<script>` in the `<head>` of `index.html`, executed before any CSS renders:

```
<head>
  <script>
    const theme = localStorage.getItem('studyhub_theme') || 'dark';
    document.documentElement.setAttribute('data-theme', theme);
  </script>
  <link rel="stylesheet" href="/css/variables.css">
  ...
</head>
```

By setting `data-theme` on the `<html>` element before the stylesheets load, the correct theme tokens are in place before any element is painted. No flash. This is a known pattern (sometimes called a "blocking script" for theme detection) that trades a tiny synchronous script execution for a flicker-free experience.

Section 05 — JavaScript: The Single-Language Stack Decision

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

5.1 The Language Is Not the Architecture

Choosing JavaScript as the sole language across the stack was the first decision. What that JavaScript would look like — how it would be structured, what patterns it would use, how asynchrony would be handled — was a separate and equally important set of decisions.

This section documents the JavaScript architectural choices at the foundational level: asynchrony patterns, module organization, the IIFE module system, and the core utilities that every other module depends on.

5.2 Async/Await Over Callbacks and Raw Promises

JavaScript's history of asynchrony is well-documented: callback pyramids, then Promise chains, then `async/await`. By the time Study-Hub was built, `async/await` was universally supported and universally preferred.

I used `async/await` exclusively throughout both the frontend and backend. The code reads synchronously even when it is not:

```
const getNotes = async (req, res, next) => {
  try {
    const result = await query('SELECT * FROM notes WHERE user_id = $1', [req.user.id]);
    res.json({ success: true, notes: result.rows });
  } catch (err) {
    next(err);
  }
};
```

The `try/catch` around `await` expressions is the standard error-handling pattern. When an awaited promise rejects, the rejection becomes a thrown exception that the `catch` block handles. In Express route handlers, `next(err)` passes the error to the error-handling middleware — it does not crash the server.

I made one decision early that saved me from repetitive code: every controller method passes its caught error to `next(err)` rather than calling `res.status(500).json(...)` inline. This keeps error *handling* logic centralized in `middleware/errorHandler.js` and keeps error *reporting* distributed at the point of failure.

5.3 The IIFE Module Pattern

The frontend could not use ES Modules (`import/export`) without a build tool or a server configured to serve the correct MIME type for `.mjs` files. I chose neither. Instead, I used the classic Immediately Invoked Function Expression (IIFE) pattern to create encapsulated modules that export a single global object:

```
// storage.js
const Storage = (() => {
  const PREFIX = 'studyhub_';

  // Private - not accessible from outside
  const _serialize = (value) => JSON.stringify(value);
  const _deserialize = (str) => JSON.parse(str);

  // Public interface
  const set = (key, value) => {
    try {
      localStorage.setItem(PREFIX + key, _serialize(value));
      return true;
    } catch (e) {
      console.warn('Storage.set failed:', e);
      return false;
    }
  };

  const get = (key, fallback = null) => {
```

```
    try {
      const raw = localStorage.getItem(PREFIX + key);
      return raw !== null ? _deserialize(raw) : fallback;
    } catch (e) {
      return fallback;
    }
  };

  return { set, get, remove, getToken, setToken, ... };
})();
```

The IIFE executes immediately, creating a local scope. `_serialize` and `_deserialize` are private — they exist inside the function scope but are not on the returned object and cannot be accessed from outside. The returned object (`{ set, get, ... }`) becomes the `Storage` global.

This pattern gives you: - **Encapsulation:** private state and helper functions that cannot be accidentally mutated from outside - **Clear public interface:** the returned object is an explicit contract - **No build step:** plain JavaScript that any browser understands

The cost is that globals pollute the `window` object: `window.Storage`, `window.API`, `window.Notes`, and so on all exist. This would be a problem at scale, but for an eight-module application it is manageable. Each module name is deliberately namespaced to avoid conflicts.

5.4 The `storage.js` Module: Lessons From Getting It Wrong

My first version of `storage.js` was a thin wrapper that just added a prefix:

```
// Initial version - too naive
const Storage = {
  set: (key, value) => localStorage.setItem('sh_' + key, value),
  get: (key) => localStorage.getItem('sh_' + key),
};
```

This broke immediately when I tried to store an object — `localStorage` only stores strings. Storing `{ noteIds: ['id1', 'id2'] }` produced `"[object Object]"` in storage, which was unreadable on retrieval.

The fix was adding `JSON.stringify`/`JSON.parse`:

```
set: (key, value) => localStorage.setItem('sh_' + key, JSON.stringify(value)),
get: (key, fallback) => {
  const raw = localStorage.getItem('sh_' + key);
  return raw !== null ? JSON.parse(raw) : fallback;
}
```

Then the second breakage: in private/incognito browsing, `localStorage` throws an exception when you call `setItem`. Without `try/catch`, this crashed the entire application initialization. The final version wraps every `localStorage` call in `try/catch` and returns a sensible fallback (`null` for `get`, `false` for `set`) instead of throwing.

The lesson: wrapper modules are not "just pass-through code." The wrappers need to handle edge cases that the underlying API does not. Every I/O boundary needs defensive error handling.

5.5 The `api.js` Module: One HTTP Layer to Rule Them All

All HTTP communication with the backend flows through a single `api.js` module. I did not scatter `fetch()` calls throughout the codebase. This was a deliberate architectural constraint.

The core is a `request()` function that wraps `fetch()`:

```
const request = async (method, path, body = null) => {
  const headers = { 'Content-Type': 'application/json' };
  const token = Storage.getToken();
  if (token) headers['Authorization'] = `Bearer ${token}`;

  const config = {
    method,
    headers,
    body: body ? JSON.stringify(body) : undefined,
  };

  const response = await fetch(path, config);
  const data = await response.json();

  if (!response.ok) {
    if (response.status === 401) {
      // Token expired or invalid - force logout
      Storage.clear();
      window.location.reload();
    }
    throw new Error(data.error || `HTTP ${response.status}`);
  }

  return data;
};
```

Every API call benefits from: - **Automatic token injection:** the JWT is added to every request without the calling module knowing about it - **Automatic 401 handling:** if the server returns 401 (Unauthorized), the local storage is cleared and the page reloads to the login screen — the user is logged out without any module needing to implement logout logic - **Consistent error throwing:** the `throw new Error(data.error)` means every calling module can use `try/catch` uniformly

On top of `request()`, the public API of the module is a structured object:

```
const API = {
  auth: {
    login:    (body) => request('POST', '/api/auth/login', body),
    register: (body) => request('POST', '/api/auth/register', body),
    profile:  ()     => request('GET',  '/api/auth/profile'),
  },
  notes: {
    getAll:   (params) => request('GET', `/api/notes?${new URLSearchParams(params)}`),
    getOne:   (id)     => request('GET', `/api/notes/${id}`),
    create:   (body)   => request('POST', '/api/notes', body),
    update:   (id, b)  => request('PUT', `/api/notes/${id}`, b),
    delete:   (id)     => request('DELETE', `/api/notes/${id}`),
  },
  // ...pdfs, chat...
};
```

This structure means that from any module, calling `API.notes.create({ title: 'Test' })` is self-documenting. There is no need to remember the endpoint URL, the HTTP method, or the authentication header — the `api.js` module handles all of that.

5.6 The Event System for Cross-Module Communication

Frontend modules are loaded independently and do not import each other. `pdfs.js` does not know that `chat.js` exists. Yet there are scenarios where modules need to trigger actions in other modules: the PDF viewer has an "Ask AI" button that should switch to the Chat view and attach the current PDF.

I solved this with browser custom events, dispatched and listened for on the `window` object:

```
// pdfs.js - dispatching
const askAIAboutPDF = (pdf) => {
  window.dispatchEvent(new CustomEvent('chat:attach-pdf', { detail: { pdf } }));
  window.dispatchEvent(new CustomEvent('nav:switch',     { detail: { view: 'chat' } }));
};

// chat.js - receiving
window.addEventListener('chat:attach-pdf', (e) => {
  attachPDFToContext(e.detail.pdf);
});

// app.js - receiving navigation events
window.addEventListener('nav:switch', (e) => {
  if (e.detail?.view) switchView(e.detail.view);
});
```

This is the publish/subscribe pattern implemented with the browser's built-in event system. The benefits: modules are fully decoupled. `pdfs.js` does not need to know where `chat.js` is in the load order, whether it is initialized, or what its API looks like. It fires an event and lets the event system route it.

The cost: the event contract (event name, `detail` shape) is implicit. There is no compile-time check that the event a module dispatches matches what another module listens for. A typo in an event name produces a silently ignored event. I mitigated this by defining all event name strings as constants in `helpers.js` and using those constants consistently.

5.7 The `helpers.js` Module: Pure Utilities

`helpers.js` contains functions with no side effects and no external dependencies — pure utilities that transform input to output:

- `escHtml(str)` — escapes HTML special characters to prevent XSS
- `formatDate(isoString)` — converts ISO date strings to human-readable relative time ("2 hours ago")
- `renderMarkdown(text)` — the custom Markdown renderer (described in Section 04)
- `generateId()` — client-side UUID generation for optimistic UI updates
- `debounce(fn, delay)` — rate-limits function calls for search input handling

The `debounce` function is worth noting:

```
const debounce = (fn, delay) => {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
};
```

I used debouncing on the notes search input to avoid sending an API request on every keystroke. The search fires only when the user pauses typing for 300ms. Without this, typing "machine learning" would trigger twelve separate API calls. With debouncing, it triggers one — after the user finishes.

This pattern is particularly important for performance: even on fast hardware, twelve database queries triggered by a single search action adds unnecessary load and introduces a race condition where responses from earlier queries could arrive after responses from later queries, producing incorrect search results.

Section 06 — Codebase Architecture: Separation of Concerns in Practice

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

6.1 Why Structure Deserves Its Own Section

The structure of a codebase is not just housekeeping. It encodes a theory of the software — which things change together, which things are independent, where boundaries should be drawn. Getting the structure wrong means that every change requires touching multiple files, every bug fix risks breaking something adjacent, and every new feature has to discover where it belongs.

I spent time upfront designing the folder structure before writing a single line of application code. This section documents the decisions embedded in that structure.

6.2 The Top-Level Split: Backend and Frontend

```
study-hub/
  backend/    ← Server code (runs on the server, never sent to browser)
  frontend/   ← Client code (sent to browser, runs there)
  scripts/    ← One-time utility scripts (setup, DB initialization)
  uploads/    ← User-uploaded files (not in Git)
```

The backend/frontend split is the most fundamental division in the codebase. It enforces the conceptual separation between server-side code (which has access to the database, environment variables, and the filesystem) and client-side code (which has access to none of these).

This split also has a security implication: the server never sends backend code to the browser. When Express serves the `frontend/` folder as static files, only files inside `frontend/` are accessible via HTTP. A request for `/backend/config/database.js` would be a 404 — that

path does not exist inside `frontend/`. The database password, JWT secret, and API keys in `backend/config/database.js` cannot be accidentally exposed.

6.3 Backend Architecture: The Four-Layer Model

Within `backend/`, I used a layered architecture:

```
backend/
  config/       ← Configuration (database connection, schema)
  controllers/  ← Business logic (what to do with a request)
  middleware/   ← Request pipeline (auth, error handling, file upload)
  routes/       ← URL-to-controller mapping
  services/     ← Reusable operations (AI API, PDF parsing)
```

Each layer has a single, well-defined responsibility:

`config/` — Things that configure the application. `database.js` creates and exports the PostgreSQL connection pool. `schema.sql` defines the database tables with `CREATE TABLE IF NOT EXISTS` — safe to re-run on every server start. These files should never need to change at runtime.

`controllers/` — The business logic. Each controller handles one feature domain. `notesController.js` handles everything related to notes. `pdfController.js` handles everything related to PDFs. Controllers do not know about each other. They receive a validated, authenticated request (`req.user` is set by middleware) and return a response.

`middleware/` — Code that runs between a request arriving and a controller handling it. The auth middleware (`auth.js`) verifies the JWT token. If it is invalid, the request is rejected before the controller runs. This means controllers never receive unauthenticated requests — they can trust that `req.user` is always a real user.

`routes/` — A single file (`index.js`) that maps URL paths and HTTP methods to controller functions. It is the "directory" of the API. Reading it gives a complete picture of every endpoint in the application. Nothing in `routes/index.js` contains business logic — it only wires.

`services/` — Complex operations that are reused across controllers. `llmService.js` encapsulates all AI API communication. `pdfService.js` encapsulates PDF text extraction. These are here because they represent standalone capabilities that might be called from multiple controllers and are complex enough to warrant isolation.

6.4 The Decision to Have a `routes/index.js` File

An alternative I considered: putting route definitions directly in `server.js`. Some Express tutorials do this for small applications. I rejected it because it conflates two concerns: `server.js` configures the server infrastructure (middleware, error handlers, startup logic), while routes define the API surface. Mixing them makes `server.js` long and harder to navigate.

By extracting routes to `routes/index.js`, the relationship between URL paths and controller functions is visible in one place. When I needed to add a new endpoint, I added two lines: one in `routes/index.js` and one function in the relevant controller. The boundary was clear.

6.5 Frontend Architecture: Utils-First Dependency Order

```
frontend/
  index.html
  css/          ← Style files
  js/
      utils/    ← Low-level utilities (no dependencies)
          storage.js
          helpers.js
          api.js
      modules/  ← Feature modules (depend on utils)
          auth.js
          notes.js
          pdfs.js
          chat.js
      app.js    ← Entry point (depends on everything)
```

The dependency direction is strictly one-way: `utils` ← `modules` ← `app.js`. A utility never depends on a module. A module never depends on `app.js`. `app.js` is the only file that can depend on everything.

This matters for the script load order in `index.html`:

```html
<script src="/js/utils/storage.js"></script>
<script src="/js/utils/helpers.js"></script>
<script src="/js/utils/api.js"></script>
<script src="/js/modules/auth.js"></script>
<script src="/js/modules/notes.js"></script>
<script src="/js/modules/pdfs.js"></script>
<script src="/js/modules/chat.js"></script>
<script src="/js/app.js"></script>
```

If `app.js` loaded first, it would call `API.notes.getAll()` before `api.js` had defined the `API` object. The browser would throw `ReferenceError: API is not defined` and the application would crash silently on startup.

I discovered this in practice: early in development, I added a module to the bottom of the script list without thinking about dependencies, and the application failed to initialize. The error was not obvious in the console because it appeared during a setup step that ran before any user interaction. After that, I was meticulous about load order.

6.6 Environment Variables and the `.env` Pattern

The `.env` file pattern separates code from configuration. The `.env` file contains sensitive values:

```
NODE_ENV=development
PORT=3000
DB_HOST=localhost
DB_PASSWORD=mysecretpassword
JWT_SECRET=a_long_random_string_here
OPENROUTER_API_KEY=sk-or-v1-...
```

These are loaded at server startup by `require('dotenv').config()` — the first line of `server.js`. After this call, every value is available via `process.env.DB_PASSWORD`, `process.env.JWT_SECRET`, and so on.

The `.env` file is in `.gitignore`. It was never committed to the repository. When I deployed to Render.com, I added these values through Render's environment variable UI, where they are stored encrypted.

The `.env.example` file — which is committed — shows the structure without real values:

```
NODE_ENV=development
PORT=3000
DB_PASSWORD=your_db_password_here
JWT_SECRET=change_this_to_a_long_random_string
OPENROUTER_API_KEY=your_key_here
```

This serves as documentation: anyone setting up the project sees exactly what they need to configure.

The risk I was guarding against: committing real API keys or database passwords to a public GitHub repository. This happens more often than developers admit — GitHub has scanners that detect and alert on accidentally committed secrets, which suggests the problem is common enough to warrant automated tooling.

6.7 The `uploads/` Directory and the Ephemeral Disk Discovery

PDF files are stored in `uploads/pdfs/{userId}/{uuid}.pdf`. The directory is created on first use:

```
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}
```

This directory is in `.gitignore` — uploaded files are user data, not code, and belong in neither Git nor the repository.

In development, this works perfectly. Files persist on the local machine across server restarts.

In production on Render.com's free tier, I discovered something I had not anticipated: **the disk is ephemeral**. When Render redeploys the service (which happens on every code push, and also on routine platform maintenance), the container is replaced with a fresh one. The `uploads/` directory on the old container is gone. New PDFs uploaded after redeploy accumulate normally, but all previously uploaded files are permanently lost.

I discovered this when I pushed a CSS fix, Render redeployed, and a user reported that their uploaded PDFs had disappeared. The database records were intact — the metadata (filename, extracted text, user ID) was still there. But the actual PDF files on disk were gone.

The correct production solution is object storage (Amazon S3, Cloudflare R2, or similar) — a separate, persistent file store that survives container restarts. This costs money, which conflicted with the free-infrastructure constraint.

My response to this discovery is documented in Section 16. The short version: I documented the limitation explicitly, implemented a workaround that preserves the extracted text (so AI features continue working even if the original PDF is gone), and flagged cloud object storage as the first upgrade path when budget allows.

6.8 The `scripts/` Directory: Operational Tooling

Three scripts handle operational tasks outside the server itself:

- `scripts/setup.js` — an interactive wizard that validates the `.env` file, tests the database connection, and runs the schema initialization. First-time setup for new developers.
- `scripts/db-init.js` — applies the SQL schema. Safe to run repeatedly because every `CREATE TABLE` uses `IF NOT EXISTS`.
- `scripts/db-reset.js` — drops all tables and recreates them. Destructive, development-only.

Having these as explicit, named scripts rather than ad-hoc SQL commands meant that the operational procedures were documented in code. "How do I reset the database?" has a concrete answer: `npm run db:reset`. It also meant I could not accidentally run a destructive command by typing it directly into a terminal.

Section 07 — Server Design: Express Middleware Pipeline & Security

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

7.1 The Server as a Pipeline

The mental model I found most useful for Express is a pipeline. A request enters the top, flows through a series of processing stages, and eventually exits the bottom as a response. Each stage (middleware) can inspect and modify the request and response, pass control to the next stage, or terminate the pipeline by sending a response.

Every design decision in `server.js` is a decision about what stages belong in the pipeline, in what order, and why.

7.2 The Startup Sequence Problem

The first thing I got wrong was server startup. My initial implementation started listening for HTTP connections before verifying the database was reachable:

```
// Initial naive version
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
// database connection happens lazily when first request arrives
```

This produced a confusing failure mode: the server started successfully, but the first API request returned a 500 error because the database connection pool had not yet established. The error logged to the console was a database connection error, but the user saw a generic server error with no indication of the real cause.

I fixed this by making startup sequential and gated:

```
const start = async () => {
  const dbOk = await testConnection();
  if (!dbOk) {
    console.error('Cannot start without database. Check your .env configuration.');
    process.exit(1);  // Exit with error code - do not start a broken server
```

```
  }

  // Apply schema (safe - uses IF NOT EXISTS)
  const schema = fs.readFileSync('./config/schema.sql', 'utf-8');
  await pool.query(schema);

  app.listen(PORT, () => {
    console.log(`Study-Hub running at http://localhost:${PORT}`);
  });
};

start().catch(err => {
  console.error('Fatal startup error:', err);
  process.exit(1);
});
```

The `process.exit(1)` is intentional: a server that starts without a database connection will fail every protected request. Better to fail loudly at startup than to start silently and fail at runtime. Exit code `1` (non-zero) signals to process managers and deployment systems that startup failed — Render.com, for example, will mark the deployment as failed and not route traffic to it.

7.3 Security Headers: The Helmet Configuration

I applied Helmet as the first middleware in the pipeline because it adds security headers to the HTTP response. The most important header is the Content Security Policy (CSP).

CSP is a browser security mechanism that limits what resources the page can load. Without a CSP, any script injected into the page — through an XSS vulnerability or a compromised CDN — can run with full access to the DOM and user data. With a strict CSP, the browser refuses to run scripts from unapproved sources.

The CSP I configured:

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc:  ["'self'", "'unsafe-inline'", "https://cdnjs.cloudflare.com"],
      styleSrc:   ["'self'", "'unsafe-inline'", "https://fonts.googleapis.com"],
      fontSrc:    ["'self'", "https://fonts.gstatic.com"],
      imgSrc:     ["'self'", "data:", "blob:"],
      frameSrc:   ["'self'", "blob:"],
      workerSrc:  ["'self'", "blob:"],
      objectSrc:  ["'self'", "blob:"],
      connectSrc: ["'self'"],
    }
  }
}));
```

The `'unsafe-inline'` entry on `scriptSrc` is a CSP weakening that I accepted reluctantly. Inline scripts — JavaScript written directly in `<script>` tags in the HTML rather than in separate `.js` files — are blocked by strict CSP unless explicitly allowed. Study-Hub uses one inline script (the theme-detection script in `<head>` described in Section 04). Eliminating it would require a cryptographic nonce on the script tag, regenerated per-response, which added more complexity than it was worth at this stage.

`blob:` in `frameSrc` and `workerSrc` is required for the PDF viewer. The browser's built-in PDF renderer uses blob URLs internally. Without `blob:` in the CSP, the PDF iframe would be blocked.

7.4 The CORS Configuration and the Bug It Caused

CORS (Cross-Origin Resource Sharing) was the source of the first real bug I hit after initial deployment.

In development, I used `origin: true` — allow all origins. This is appropriate locally, where the frontend and backend run on the same `localhost:3000` and CORS is not a concern.

In production, I specified explicit allowed origins via an environment variable:

```
const allowedOrigins = process.env.ALLOWED_ORIGINS
  ? process.env.ALLOWED_ORIGINS.split(',').map(o => o.trim())
  : ['http://localhost:3000'];

app.use(cors({
  origin: process.env.NODE_ENV === 'development' ? true : allowedOrigins,
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));
```

The bug: I had set `ALLOWED_ORIGINS` in Render's environment variables to `https://study-hub.onrender.com` but the actual deployed URL was `https://study-hub-abc123.onrender.com` — Render appends a random suffix to service names. Every API call from the frontend returned a CORS error. The browser rejected all responses because the `Origin` header (`https://study-hub-abc123.onrender.com`) did not match any entry in `allowedOrigins`.

The fix was straightforward — update `ALLOWED_ORIGINS` to the correct URL — but the debugging took longer than it should have because CORS errors in the browser console do not tell you what the server received or why it rejected the request. I had to read the server logs on Render's dashboard to see the origin mismatch.

The lesson: always verify the exact deployed URL before configuring CORS. Copy it from the browser address bar, do not type it from memory.

7.5 Rate Limiting: Two Different Limiters for Two Different Threats

I applied two rate limiters with different parameters for different threat models:

```
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 10,                    // 10 attempts per 15 minutes
  message: { success: false, error: 'Too many login attempts, please try again later' }
});

const generalLimiter = rateLimit({
  windowMs: 60 * 1000,  // 1 minute
  max: 300,             // 300 requests per minute
});

app.use('/api/auth/login',    authLimiter);
```

```
app.use('/api/auth/register', authLimiter);
app.use('/api/', generalLimiter);
```

The auth limiter targets brute-force attacks against the login endpoint. At 10 attempts per 15 minutes, an attacker can try 960 password guesses per day — for a modern bcrypt-hashed password with a work factor of 12, this is computationally irrelevant. The realistic dictionary of "common passwords" has millions of entries, which would take years to exhaust at this rate.

The general limiter targets denial-of-service attempts. 300 requests per minute is generous enough for normal use (a user navigating the app generates far fewer than 5 requests per second) but low enough to prevent a script from hammering the server with thousands of requests.

I placed the general limiter after the auth limiter in the code. Both limiters apply to auth routes (the auth limiter is more restrictive and fires first), while only the general limiter applies to all other API routes.

7.6 The SPA Fallback Route

```
app.get('*', (req, res) => {
  if (req.path.startsWith('/api/')) {
    return res.status(404).json({ success: false, error: 'API route not found' });
  }
  res.sendFile(path.join(__dirname, '../frontend/index.html'));
});
```

This catch-all route is the mechanism that makes the Single-Page Application work correctly with browser navigation.

When a user visits `https://study-hub.onrender.com/notes` directly (by typing it, bookmarking it, or refreshing the page while on that view), the browser sends `GET /notes` to the server. There is no server-side route for `/notes` — that path only makes sense to the frontend JavaScript. Without the catch-all, the server would return a 404.

The catch-all intercepts every unmatched GET request and returns `index.html`. The browser then loads the SPA, which reads `window.location.pathname` (`/notes`), and navigates to the Notes view — correctly.

The `req.path.startsWith('/api/')` guard ensures that unrecognised API calls still return proper JSON 404 errors rather than the HTML file. An API consumer receiving HTML when it expected JSON would produce confusing errors.

7.7 Error Handler Middleware

Express's error handling middleware has a four-parameter signature that distinguishes it from regular middleware:

```
const errorHandler = (err, req, res, next) => {
  console.error('Unhandled error:', err.stack);

  const status  = err.status || err.statusCode || 500;
  const message = err.message || 'Internal Server Error';

  // Don't expose internal error details in production
```

```
  const response = {
    success: false,
    error: process.env.NODE_ENV === 'production'
      ? (status < 500 ? message : 'An internal error occurred')
      : message,
  };

  res.status(status).json(response);
};
```

The four-parameter signature (`err, req, res, next`) is how Express identifies an error handler — it will not be called as regular middleware. When any route handler calls `next(err)`, Express skips all remaining regular middleware and jumps directly to this function.

The production/development distinction in the error message is a security consideration: in production, 500-level errors (server crashes) return a generic message to avoid leaking implementation details (database schema, file paths, package names) that an attacker could use. 4xx errors (client errors) are safe to expose — they describe what the client did wrong, not what the server looks like internally.

7.8 What I Would Add if Not For the Free Tier

Two security additions I chose not to implement due to infrastructure constraints:

Redis for session/token blacklisting. JWTs cannot be revoked. If a user changes their password, the old token remains valid until expiry. A Redis-based token blacklist would allow immediate invalidation. Redis requires a running cache service — available managed, but not on a fully free tier.

Request signing. For high-security scenarios, HMAC-signed requests would prevent replay attacks. Overkill for this application, but worth noting as a gap.

Both would be straightforward additions on a paid infrastructure tier. They are architectural gaps I accepted consciously, not oversights.

Section 08 — Data Modelling: Schema Design & PostgreSQL Selection

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

8.1 The Data Model as the Foundation

In my experience, the database schema is the decision that is hardest to change later. Frontend code can be rewritten. API endpoints can be versioned. But a schema migration on a live database with user data requires careful planning, backward compatibility consideration, and usually downtime. Getting the schema right early saves significant pain later.

I spent more time designing the schema before writing any application code than I spent on any other design task.

8.2 Why UUID Primary Keys Over Serial Integers

The most visible schema decision is the use of UUID (`uuid` type) as the primary key for every table rather than auto-incrementing integers (`SERIAL` / `BIGSERIAL`).

The conventional choice is integers: `id SERIAL PRIMARY KEY`. They are compact, fast to index, and easy to read. Most tutorial databases use them.

I chose UUIDs for three reasons:

Security. Integer IDs are sequential and predictable. If a user's note has `id = 42`, an attacker who knows the URL pattern (`/api/notes/42`) can guess that notes 41 and 43 also exist and attempt to access them. UUIDs are 128-bit random values — guessing a valid UUID is computationally infeasible. This is defense-in-depth: ownership checks at the query level (`AND user_id = $2`) already prevent unauthorized access, but UUIDs eliminate even the information leak of knowing that an ID exists.

Distributed generation. UUIDs can be generated on the client side before a record is created on the server. This enables optimistic UI patterns (showing the note card before the server responds) without a round-trip to discover the ID.

Merge safety. If two databases ever need to be merged (e.g., during a database migration), UUID primary keys never conflict. Integer IDs from two separate databases will collide. For Study-Hub's current scale this does not matter, but it is a good habit.

The cost: UUIDs are 36 characters (`8-4-4-4-12` format), larger than a 4-byte integer. Index sizes are larger. This is measurable but negligible at the scale Study-Hub operates at.

8.3 The Core Schema: Seven Tables

The Study-Hub schema consists of seven tables:

`users` — Accounts. Core fields: `id` (UUID), `name`, `email` (unique), `password_hash`, `avatar_color`, `created_at`.

`note_folders` — Organizational containers for notes. Each folder belongs to a user (`user_id` foreign key). Fields: `id`, `user_id`, `name`, `icon`, `color`, `created_at`.

`notes` — The core content objects. Each note belongs to a user and optionally to a folder. Fields: `id`, `user_id`, `folder_id` (nullable), `title`, `content` (TEXT), `color`, `is_pinned` (BOOLEAN), `is_archived` (BOOLEAN), `tags` (TEXT[]), `created_at`, `updated_at`.

`pdf_folders` — Organizational containers for PDFs, mirroring the note folder structure.

`pdfs` — Uploaded PDF metadata. The actual file bytes are on disk. Fields: `id`, `user_id`, `folder_id` (nullable), `filename` (the UUID on-disk name), `original_name` (the user's original filename), `file_size`, `page_count`, `extracted_text` (TEXT — the full text of the PDF), `created_at`.

`chat_sessions` — Named conversation threads. Each session belongs to a user. Fields: `id`, `user_id`, `title`, `created_at`, `updated_at`.

`chat_messages` — Individual messages within sessions. Fields: `id`, `session_id`, `user_id`, `role` (`'user'` or `'assistant'`), `content`, `context_type`, `context_ref_id`, `created_at`.

8.4 The Tags Column: PostgreSQL Arrays

Notes support multiple tags. The natural relational approach is a separate `note_tags` table:

```
CREATE TABLE note_tags (
  note_id UUID REFERENCES notes(id) ON DELETE CASCADE,
  tag     TEXT NOT NULL
);
```

This is correct in theory. But it means every note fetch requires a JOIN to `note_tags`, and every tag update requires either deleting and reinserting rows or a complex merge operation.

I used PostgreSQL's native array type instead:

```
tags TEXT[] DEFAULT '{}'
```

This stores an array of strings in a single column. Querying is straightforward:

```
-- Get notes with a specific tag
SELECT * FROM notes WHERE 'machine-learning' = ANY(tags);

-- Update tags
UPDATE notes SET tags = $1 WHERE id = $2;
-- $1 is passed as a JavaScript array: ['machine-learning', 'ai']
```

The tradeoff: PostgreSQL arrays are not relational. You cannot easily build a "all tags used by user X" query without unnesting. I accepted this because tags in Study-Hub are relatively simple — they are added and removed from notes, not queried globally. If I ever needed tag analytics ("show me all tags I've ever used across all notes"), a separate table would be necessary.

8.5 Full-Text Search: to_tsvector and to_tsquery

The notes search feature uses PostgreSQL's built-in full-text search rather than a separate search index (like Elasticsearch or Meilisearch, both of which cost money or infrastructure overhead).

The search query:

```
SELECT n.*, f.name AS folder_name
FROM notes n
LEFT JOIN note_folders f ON n.folder_id = f.id
WHERE n.user_id = $1
  AND n.is_archived = false
  AND to_tsvector('english', COALESCE(n.title, '') || ' ' || COALESCE(n.content, ''))
      @@ to_tsquery('english', $2)
ORDER BY ts_rank(
  to_tsvector('english', COALESCE(n.title, '') || ' ' || COALESCE(n.content, '')),
  to_tsquery('english', $2)
) DESC;
```

`to_tsvector('english', text)` converts text to a search document — tokenizing it, removing stop words, and reducing words to their stems ("learning" → "learn"). The `'english'` argument selects the language-specific dictionary.

`to_tsquery('english', $2)` converts the search term to a query. The `@@` operator checks whether the document matches the query.

`ts_rank(...)` scores how relevant each document is to the query. The `ORDER BY ts_rank ... DESC` sorts results by relevance, putting the most relevant notes first.

`COALESCE(n.title, '') || ' ' || COALESCE(n.content, '')` concatenates the title and content with a space. `COALESCE` handles NULL values — if either field is NULL (a note with no title, for example), it substitutes an empty string rather than producing NULL, which would break the concatenation.

The limitation of this approach: it is computed at query time, not pre-indexed. For large volumes of notes, this would be slow. A proper implementation would maintain a GIN index (`CREATE INDEX ON notes USING GIN(to_tsvector('english', title || ' ' || content))`) to avoid recomputing vectors on every search. I initially omitted this index and later added it when I noticed search latency on a test dataset with a few hundred notes.

8.6 The Dynamic Query Builder: A Design Problem I Fought With

The notes listing endpoint supports multiple simultaneous filters: `folder`, `search`, `archived`, `pinned`. These are all optional. The SQL query needs to include only the `WHERE` clauses that are relevant to the request.

My initial approach used string concatenation to build the query dynamically:

```
// Initial, wrong approach
let sql = 'SELECT * FROM notes WHERE user_id = $1';
if (folder) sql += ` AND folder_id = '${folder}'`;  // SQL INJECTION VULNERABILITY
```

This is a SQL injection vulnerability. If `folder` contains `' OR '1'='1`, the WHERE clause becomes `AND folder_id = '' OR '1'='1'` — which is always true and returns all notes for all users.

The correct approach uses parameterized queries with dynamic parameter indexing:

```
const values = [req.user.id];
let idx = 2;
const conditions = ['n.user_id = $1'];

if (folder === 'uncategorized') {
  conditions.push('n.folder_id IS NULL');
} else if (folder) {
  conditions.push(`n.folder_id = $${idx++}`);
  values.push(folder);
}

if (search) {
  conditions.push(`to_tsvector('english', ...) @@ to_tsquery('english', $${idx++})`);
  values.push(search.split(' ').filter(Boolean).join(' & '));
}

const sql = `SELECT ... FROM notes n WHERE ${conditions.join(' AND ')} ORDER BY ...`;
await query(sql, values);
```

`idx++` is the key pattern: each new parameter gets the next number in sequence, and `idx` increments for the next one. The `values` array is built in parallel — position `values[0]` is `$1`, `values[1]` is `$2`, and so on. The database driver maps parameters by position, never interpreting the values as SQL.

I initially missed that the `search` input needed to be sanitized for the `to_tsquery` format. A search for "machine learning" needs to be converted to `machine & learning` (the `&` operator

means “both words must appear”). Spaces between words are not automatically treated as AND in PostgreSQL full-text query syntax. I found this when a two-word search returned zero results and I spent an hour debugging the query before realizing the query format was wrong.

8.7 Connection Pooling and the Pool Configuration

I used the `pg` library’s connection pool rather than creating a new database connection on each request:

```
const { Pool } = require('pg');
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production'
    ? { rejectUnauthorized: false }
    : false,
  max: 10,          // maximum connections in pool
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});
```

A database connection is an expensive resource — establishing the TCP connection, performing TLS handshake, sending credentials, and completing the PostgreSQL authentication protocol takes tens of milliseconds. Doing this per-request would make every API call 50-100ms slower.

The pool maintains up to 10 open connections. Incoming requests acquire a connection from the pool, perform their query, and release the connection back to the pool. If all 10 are in use, new requests wait up to `connectionTimeoutMillis` (2 seconds) before throwing an error.

`ssl: { rejectUnauthorized: false }` in production is a known Render.com requirement. Render’s managed PostgreSQL uses SSL but with a self-signed certificate. `rejectUnauthorized: false` disables certificate validation, which is a security tradeoff — it allows connecting over SSL without verifying the server’s certificate chain. In a higher-security environment, I would provision and trust the server’s root certificate instead.

Section 09 — Authentication: bcrypt, JWT, and the Statelessness Decision

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

9.1 The Authentication Problem

Authentication answers one question: “Is this request coming from who it claims to be?” For a multi-user application where every piece of data is private to its owner, answering this question incorrectly is a catastrophic failure. I treated authentication as one of the highest-risk subsystems in the application and made decisions accordingly.

This section documents three separate decisions: how passwords are stored, how identity is proven on each request, and how the server verifies claims without database lookups.

9.2 Password Storage: bcrypt at Cost Factor 12

The invariant rule in web application security is: **never store passwords in plaintext.** If the database is ever compromised, plain-text passwords expose every user's account — and because users reuse passwords, their email, bank accounts, and other services are at risk too.

The standard approach is hashing. A cryptographic hash function produces a fixed-length output from arbitrary input, with two key properties: it is deterministic (same input always produces the same output) and computationally irreversible (you cannot recover the input from the output).

I chose bcrypt over SHA-256, MD5, and other general-purpose hash functions for a specific reason: **bcrypt is deliberately slow.** General-purpose hashes compute billions of outputs per second on modern hardware. At that speed, an attacker with a stolen database can try billions of password guesses per second — a dictionary of the 10 million most common passwords can be tested in under a second.

bcrypt has a configurable work factor. I set it to 12:

```
// authController.js - Registration
const passwordHash = await bcrypt.hash(password, 12);
// Work factor 12 = 2^12 = 4096 rounds of the KDF
// Each hash takes approximately 250ms on a modern CPU
```

At 250ms per hash, an attacker with the database can test approximately 345,600 passwords per day — far below the billions-per-second rate that makes dictionary attacks viable. The work factor also scales with hardware: as CPUs get faster, the work factor can be increased to maintain the same time cost.

The `12` argument is the work factor, not the number of iterations directly. The actual number of iterations is `2^12 = 4,096`. A work factor of 13 would be `2^13 = 8,192` iterations — twice as slow.

bcrypt also includes automatic salting. A salt is a random value mixed into the password before hashing:

```
salt = random("x8Pk7qL...")
hash("password123" + salt) → "$2a$12$x8Pk7qL...hashed_output"
```

The salt is stored within the hash string itself (the prefix). Without salting, users with the same password would produce the same hash, and precomputed "rainbow table" attacks could immediately reveal the password. With per-user random salts, even identical passwords produce different hashes, and rainbow tables become useless.

Login verification:

```
// authController.js - Login
const result = await query(
  'SELECT id, name, email, password_hash FROM users WHERE email = $1',
  [email.toLowerCase()]
);

if (!result.rows.length) {
  // User not found - same error as wrong password (security: don't reveal if email exists)
  return res.status(401).json({ success: false, error: 'Invalid email or password' });
}

const user = result.rows[0];
const isValid = await bcrypt.compare(password, user.password_hash);
// bcrypt.compare re-hashes the input with the stored salt and compares
```

The error message “Invalid email or password” — not “Email not found” or “Wrong password” — is intentional. Distinguishing between the two would allow an attacker to enumerate valid email addresses by observing which error message they receive.

9.3 JWT: Why Stateless Tokens

After verifying credentials, the server issues a JSON Web Token. The client stores this token and sends it with every subsequent request. The server verifies the token to authenticate future requests.

The JWT structure is three base64-encoded parts separated by periods:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.   ← Header (algorithm + type)
eyJpZCI6ImFiYy0xMjMiLCJpYXQiOjE2OTI...   ← Payload (user data + timestamps)
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_...   ← Signature (HMAC-SHA256 of header + payload)
```

The payload contains:

```
{
  "id": "user-uuid-here",
  "iat": 1692000000,   // issued at (Unix timestamp)
  "exp": 1692086400    // expires at (24 hours later)
}
```

The signature is computed with the server’s `JWT_SECRET`. Any modification to the payload invalidates the signature. The server can verify the token’s integrity by recomputing the signature and comparing — no database lookup required.

Why I chose JWT over sessions:

Sessions require a persistent store. The server generates a session ID, stores the session data in a database or cache, and sends only the session ID to the client. On each request, the server looks up the session ID to retrieve the user data. This creates a round-trip I/O cost on every protected request, and requires a cache service (Redis is typical) for performance.

JWTs are self-contained. The user’s ID is embedded in the token. The server reads it with `jwt.verify(token, secret)` — a synchronous cryptographic operation with no I/O. At the scale Study-Hub operates at, this difference is small, but it also eliminates the need for any session store infrastructure.

The known tradeoff:

JWTs cannot be revoked before expiry without additional infrastructure. If a user changes their password, the old token remains valid for its remaining lifetime. If a token is stolen, there is no server-side way to invalidate it.

I mitigated this with a 24-hour expiry: tokens are automatically invalid after one day. For password changes, I accepted the limitation: changing a password does not immediately invalidate existing sessions. For a banking application this would be unacceptable. For a personal study tool, I judged it an acceptable tradeoff.

9.4 The Authentication Middleware

The `auth.js` middleware runs before every protected route handler:

```
// middleware/auth.js
const authenticate = async (req, res, next) => {
  const authHeader = req.headers['authorization'];
  // Expected format: "Bearer eyJhbGci..."

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ success: false, error: 'No token provided' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    // decoded = { id: 'user-uuid', iat: ..., exp: ... }

    // Fetch user from database to ensure they still exist
    const result = await query(
      'SELECT id, name, email, avatar_color FROM users WHERE id = $1',
      [decoded.id]
    );

    if (!result.rows.length) {
      return res.status(401).json({ success: false, error: 'User not found' });
    }

    req.user = result.rows[0];  // Attach user to request
    next();  // Continue to route handler

  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({ success: false, error: 'Token expired' });
    }
    return res.status(401).json({ success: false, error: 'Invalid token' });
  }
};
```

There is a deliberate database query inside the middleware: `SELECT ... FROM users WHERE id = $1`. This looks inefficient — one extra query per request. But it is necessary for one edge case: if an administrator deletes a user from the database, their token remains cryptographically valid. Without the database lookup, the deleted user could still access the API until their token expires. The lookup ensures that `req.user` always represents an actually-existing user.

I could have skipped this lookup and trusted the payload, but the cost (one indexed primary key lookup per request, hitting the query cache almost every time) was low enough that the correctness guarantee was worth it.

9.5 JWT Expiry: The Headache I Hit in Production

The 24-hour expiry worked perfectly in development. In production, I discovered a UX problem: users who left Study-Hub open in a browser tab overnight would return the next day to find every action silently failing. The token had expired, but the frontend was still trying to use it.

The `api.js` module handled this:

```
// api.js
if (response.status === 401) {
  Storage.clear();
  window.location.reload();
}
```

When the server returns 401, the frontend clears localStorage and reloads the page, forcing the user back to the login screen. This is technically correct but jarring: the user clicks "Create Note," sees nothing happen, then the page reloads to login. There is no message explaining what happened.

The better solution — a token refresh mechanism, where a second long-lived "refresh token" is used to obtain a new short-lived access token — would have required additional server endpoints, secure refresh token storage (httpOnly cookies, not localStorage), and more complex client-side logic. It was the right solution, but I did not implement it in the first version.

I added a toast notification before the reload:

```
if (response.status === 401) {
  Helpers.toast('Your session expired. Logging you out...', 'warning');
  setTimeout(() => {
    Storage.clear();
    window.location.reload();
  }, 2000);
}
```

This gives the user two seconds to read the message before the reload. It is a patch, not a fix, and I note it in Section 17 as future work.

9.6 The Input Validation Layer

Before any password is hashed or token is issued, the registration handler validates inputs:

```
const { name, email, password } = req.body;

if (!name || !email || !password) {
  return res.status(400).json({ success: false, error: 'All fields are required' });
}

if (password.length < 8) {
  return res.status(400).json({ success: false, error: 'Password must be at least 8 characters' });
}

const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(email)) {
  return res.status(400).json({ success: false, error: 'Invalid email format' });
}
```

Validation happens server-side, not just client-side. Client-side validation is convenience for the user. Server-side validation is security — an attacker will bypass the frontend entirely and POST directly to the API. Trusting client-side validation and skipping server-side means the server accepts malformed input.

The email regex `^[^\s@]+@[^\s@]+\.[^\s@]+$` is a deliberately minimal check: does the string have a `@` with non-whitespace characters around it, and a `.` after the `@`? This accepts `x@y.z` (technically a valid email) and rejects obviously malformed strings. A perfect RFC 5321-compliant

email parser is significantly more complex and offers little additional value for a web form. I used the minimal check that catches practical errors.

Section 10 — REST API Design: Conventions, Ownership, and Consistency

---

10.1 Why API Design Is a Separate Decision From Implementation

It would have been easy to design the API reactively — add an endpoint when the frontend needs something, shape the response to match whatever the frontend is currently doing. Many projects are built this way. The result is an API that is internally inconsistent: some endpoints return `{ data: ... }`, some return `{ result: ... }`, some put the resource directly at the root, error formats vary across endpoints, and HTTP status codes are used inconsistently.

I made the API design decisions upfront, before writing any controller code, and documented them as invariants that every endpoint had to follow. This section describes those invariants and the reasoning behind each.

10.2 REST as a Design Style, Not a Religion

Study-Hub's API is RESTful in the conventional sense: URLs identify resources, HTTP methods identify actions, and stateless requests carry all necessary context.

The core mapping:

HTTP Method	Meaning	SQL Equivalent
`GET`	Read/retrieve	`SELECT`
`POST`	Create	`INSERT`
`PUT`	Replace/update	`UPDATE`
`PATCH`	Partial update	`UPDATE`
`DELETE`	Remove	`DELETE`

This mapping makes the API predictable. When I looked at any endpoint in the codebase, I knew what it did from the method alone before reading the handler. `DELETE /api/notes/:id` deletes a note. `GET /api/notes` retrieves notes. There is nothing to memorize because the convention is self-consistent.

I used both `PUT` and `PATCH` selectively: `PUT` for full replacements (updating a note replaces its content, title, and metadata), `PATCH` for partial state changes (archiving a note, which only changes `is_archived`). In practice the distinction mattered less than I initially anticipated — clients always sent only the fields they wanted to change — but keeping the distinction in place preserved the semantic clarity.

10.3 The Invariant Response Format

Every endpoint — success or failure — returns JSON in one of two shapes:

```
// Success
{
  "success": true,
  "notes": [ ... ]          // or "note", "user", "session", etc.
}

// Error
{
  "success": false,
  "error": "Human-readable description of what went wrong"
}
```

This invariant was non-negotiable. The `api.js` frontend module depends on it:

```
const request = async (method, path, body) => {
  const response = await fetch(path, config);
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.error || `HTTP ${response.status}`);
  }

  return data;
};
```

Because `data.error` is always the error message (when there is one), the frontend's error handling is completely uniform. In a call site:

```
try {
  const data = await API.notes.create({ title: 'New Note' });
  // data.note is available
} catch (err) {
  toast.error(err.message);  // err.message is always the human-readable error
}
```

No case analysis, no format guessing. The alternative — different error shapes from different endpoints — would require each call site to know how to parse that specific endpoint's errors. That coupling is the kind of thing that produces bugs when endpoints are modified.

10.4 HTTP Status Code Discipline

I mapped status codes consistently:

- `200 OK` — successful read, update, or delete
- `201 Created` — successful resource creation (POST that makes something new)
- `400 Bad Request` — the request is malformed (missing required fields, invalid format)
- `401 Unauthorized` — the user is not authenticated (missing or invalid token)
- `403 Forbidden` — the user is authenticated but not allowed (reserved for future admin features)
- `404 Not Found` — the resource does not exist (or the user does not own it)
- `409 Conflict` — a resource with these identifiers already exists (duplicate email on register)

- `429 Too Many Requests` — rate limit exceeded
- `500 Internal Server Error` — an unexpected exception on the server

The 404/403 distinction for authorization is a deliberate choice. When a user requests a note they do not own, my queries use `WHERE id = $1 AND user_id = $2`. If the note exists but belongs to someone else, the query returns 0 rows — the controller returns 404. The user cannot distinguish "this note does not exist" from "this note exists but you cannot see it." Returning 403 would confirm to an attacker that the resource exists, which is information leakage.

10.5 Ownership Enforcement at the Query Level

Every query that reads or modifies a resource includes an explicit ownership check as part of the `WHERE` clause:

```
-- Getting a specific note
SELECT n.*, f.name AS folder_name
FROM notes n
LEFT JOIN note_folders f ON n.folder_id = f.id
WHERE n.id = $1 AND n.user_id = $2;
--           ↑ resource ID    ↑ must be the logged-in user
```

`$2` is always `req.user.id` — set by the authentication middleware, never from the request body or URL parameters.

This is the critical security property of the API: even if an attacker knows a valid note UUID (perhaps they created a note and inspected its ID), they cannot read another user's note by substituting that UUID. The `AND user_id = $2` clause means the query will return 0 rows if the IDs do not match the same user.

I made this a code review rule for myself: no query that touches user-owned data can omit the `user_id` condition. I checked this on every query I wrote.

10.6 The Chat Endpoint: The Most Complex Route

The `POST /api/chat/sessions/:sessionId/messages` endpoint is an order of magnitude more complex than any other route. Its handler performs:

1. Validate the request body (message content, context IDs)
2. Verify session ownership (the session must belong to this user)
3. Insert the user's message into the database
4. Fetch the content of any attached notes/PDFs
5. Fetch the last 20 messages of the session
6. Call the LLM service with context and history
7. Parse the AI response for `[[CREATE_NOTE]]` tags
8. If found: create a new note, strip the tags from the response
9. Insert the AI response into the database
10. If this is the first message in the session: auto-update the session title
11. Return both messages to the client

This is a sequential chain of async operations where each step depends on the previous. The failure mode of any step needed to be handled correctly without corrupting state. I structured it as a long `async/await` chain with early returns on validation failures and a broad `catch` that captures any unhandled error.

One design decision in this endpoint: the user's message is saved to the database (step 3) before the AI call (step 6). If the AI call fails, the user's message is still recorded. The conversation history is not lost. The response will include an error message in the AI's position, but the user can retry without losing context.

The alternative — saving both messages together only after the AI responds — would mean that if the AI call times out, neither message is saved, and the user must re-type their question. Saving the user message first preserves history at the cost of an occasionally "orphaned" user message with no AI response following it. I considered this the lesser evil.

10.7 The PDF Streaming Endpoint's Auth Exception

Most endpoints authenticate via the `Authorization: Bearer <token>` header. The PDF streaming endpoint (`GET /api/pdfs/:id/stream`) authenticates via a URL query parameter:

```
GET /api/pdfs/abc-123/stream?token=eyJhbGci...
```

The reason is a browser limitation: `<iframe>` elements and native PDF viewers make bare GET requests that cannot include custom headers. The browser's PDF viewer, when given an `<iframe src="...">`, sends the request with default browser headers only. There is no JavaScript interception point to inject an `Authorization` header.

URL tokens are a security downgrade from header tokens: they appear in server access logs, browser history, and referrer headers. For a production system handling sensitive documents, the correct solution is a one-time pre-signed URL (an endpoint that generates a short-lived signed URL for a specific PDF, valid for 30-60 seconds, single-use). I chose the URL query parameter approach for simplicity and documented it as a known security tradeoff.

10.8 API Versioning: A Decision I Deferred

I did not version the API (`/api/v1/...`). All routes are under `/api/`. This means any breaking change to the API requires updating the frontend simultaneously — there is no old API version for old clients to fall back to.

For a single-page application where the frontend and backend are deployed together (as they are on Render), this is acceptable: there are no external API consumers, and frontend and backend are always in sync. Versioning becomes essential when external developers consume the API or when mobile clients (which cannot be force-updated) exist. Study-Hub has neither.

I would add versioning if Study-Hub were ever opened to external integrations.

Section 11 — SPA Architecture: Module Pattern Without a Build Tool

---

11.1 The Build Tool Question

Modern frontend development almost universally involves a build tool: Webpack, Vite, Parcel, Rollup, or one of their successors. These tools bundle multiple JavaScript files into a single optimized file, transpile modern JavaScript for older browsers, enable JSX or TypeScript, perform dead code elimination, and provide hot module replacement during development.

I chose to build Study-Hub's frontend without any of these tools. This decision shaped every other frontend architectural choice, and I want to be honest about both what it enabled and what it cost.

What it enabled: zero configuration overhead at the start, immediate feedback loop during development (edit a file, refresh the browser, see the change), outputs I could inspect directly without source maps, and no build failures to debug. When you are the only developer and you are moving fast, zero build overhead is genuinely valuable.

What it cost: no tree-shaking (unused code ships to the browser), no TypeScript (no compile-time type errors), no JSX (no component syntax), no module import/export (no ES module system), and no hot module replacement. For a small application with a single developer, these costs were acceptable. For a larger team or codebase, they would not be.

11.2 The IIFE Module Pattern in Practice

Without ES modules or a bundler, I needed another way to organize code into separate files with private state and a public interface. The solution is the IIFE (Immediately Invoked Function Expression) pattern, described in Section 05. Here I want to document how it works across an eight-module codebase.

Each module follows the same structure:

```
const ModuleName = (() => {
  // === PRIVATE STATE ===
  let state = { ... };

  // === PRIVATE HELPERS ===
  const renderSomething = () => { ... };

  // === PUBLIC INTERFACE ===
  const init = () => { ... };       // Called by app.js once
  const refresh = () => { ... };    // Called by app.js on view switch

  return { init, refresh };
})();
```

The `init()` function is the module's entry point — it attaches event listeners, sets up DOM references, and performs any one-time initialization. It is called by `app.js` after everything is loaded. The `refresh()` function is called when the user navigates to that module's view — it re-fetches data and re-renders.

`app.js` orchestrates:

```
// app.js
const initApp = async () => {
  await Auth.init();   // sets up auth UI, checks if logged in

  if (Storage.getToken()) {
```

```
    await Notes.init();
    await PDFs.init();
    await Chat.init();
    showView('notes');  // default landing view
  }
};

window.addEventListener('DOMContentLoaded', initApp);
```

`DOMContentLoaded` fires when the HTML is fully parsed but before images and external resources load. Using it as the entry point ensures that all DOM elements exist before any JavaScript tries to query them.

11.3 State Management: Plain JavaScript Objects

React and Vue have sophisticated state management systems (useState hooks, Vuex stores). Without a framework, I used plain JavaScript objects within each module's closure:

```
// notes.js (simplified)
const Notes = (() => {
  let state = {
    notes:          [],
    folders:        [],
    activeFolder:   null,
    activeNote:     null,
    searchQuery:    '',
    isLoading:      false,
    editorOpen:     false,
  };

  const setState = (updates) => {
    state = { ...state, ...updates };
    render();  // re-render after every state change
  };

  // ...
})();
```

The `setState` pattern mirrors React's mental model: you never mutate `state` directly, you call `setState` with the changes, which triggers a re-render. This makes the flow predictable: state changes always cause a re-render, and re-renders always reflect the current state.

The downside: re-rendering means calling `renderNotesList()`, which clears the container's `innerHTML` and rebuilds all DOM nodes from scratch. This is inefficient — for a list of 100 notes, all 100 note card DOM nodes are destroyed and recreated every time any single note changes. React's virtual DOM diffing avoids this by only updating the nodes that actually changed.

For Study-Hub's scale — typical users have tens to low hundreds of notes — this performance cost is invisible. Rendering 100 cards takes a few milliseconds. For thousands of notes, it would become a problem. I accepted the tradeoff as appropriate for the current scale.

11.4 DOM Querying: The `el()` Shorthand and Why I Cached References

Querying the DOM with `document.getElementById('some-id')` or `document.querySelector('.some-class')` is synchronous and fast on a single call. But when called repeatedly — in render loops, event handlers, and utility functions — it adds up to unnecessary work: the browser traverses the DOM tree on every call.

I defined a shorthand:

```
const el = (id) => document.getElementById(id);
```

And wherever an element was referenced more than once, I cached the reference at module initialization:

```
const Notes = (() => {
  // References cached once at init
  let notesContainer, searchInput, editorPanel, editorTitle, editorContent;

  const init = () => {
    notesContainer = el('notes-list-container');
    searchInput    = el('notes-search-input');
    editorPanel    = el('note-editor-panel');
    editorTitle    = el('note-editor-title');
    editorContent  = el('note-editor-content');

    attachEventListeners();
  };
})();
```

After `init()`, `notesContainer` holds a direct reference to the DOM element. Subsequent uses of `notesContainer` do not query the DOM. This is a minor optimization, but it also has a secondary benefit: if an element ID changes in the HTML, the error is thrown in `init()` (where the reference is set up) rather than somewhere unexpected in the middle of a render function.

11.5 The Optimistic UI Pattern

When a user creates a note, the naive implementation would: 1. Send a POST request 2. Wait for the server to respond 3. Update the UI with the server's response

This produces visible latency: the user clicks "Save Note," waits 200-400ms for the round trip, then sees the note appear. For fast-feedback interfaces, this latency is perceptible and frustrating.

I used an optimistic UI pattern for note creation:

```
const createNote = async (noteData) => {
  // Generate a temporary client-side ID
  const tempId = 'temp-' + Date.now();

  // 1. Add note to local state immediately (optimistic)
  const tempNote = { ...noteData, id: tempId, created_at: new Date().toISOString() };
  state.notes.unshift(tempNote);
  renderNotesList();  // User sees the note immediately

  try {
    // 2. Send to server
```

```
    const { note: serverNote } = await API.notes.create(noteData);

    // 3. Replace temp note with server's authoritative version
    state.notes = state.notes.map(n => n.id === tempId ? serverNote : n);
    renderNotesList();

  } catch (err) {
    // 4. Roll back if server rejects
    state.notes = state.notes.filter(n => n.id !== tempId);
    renderNotesList();
    toast.error('Failed to save note: ' + err.message);
  }
};
```

The user sees the note appear instantly. If the server accepts it, the temporary note is replaced with the server's version (which has the real UUID, `created_at` from the database, etc.). If the server rejects it, the optimistic note is removed and an error toast explains what happened.

The tradeoff: the brief moment when the note exists in the UI but not yet in the database. If the user immediately tries to reference the note by its ID (e.g., to attach it to a chat), the `tempId` will fail server lookups. I guarded against this by disabling interaction with cards that have `tempId` prefixes until the server response arrives.

11.6 The Typing Indicator: UX Engineering

The AI chat typing indicator was a small engineering challenge that consumed more time than expected. The naive approach — display "AI is typing…" as text — works but feels sterile.

I implemented a shimmer skeleton effect: three animated lines of different widths that pulse with a shimmer animation, giving the impression of text about to materialize:

```
const showTypingIndicator = () => {
  const el = document.createElement('div');
  el.className = 'message-wrap assistant typing-wrap';
  el.id = 'typing-indicator';
  el.innerHTML = `
    <div class="message-bubble ai-bubble shimmer-bubble">
      <div class="shimmer-line shimmer-line--long"></div>
      <div class="shimmer-line shimmer-line--mid"></div>
      <div class="shimmer-line shimmer-line--short"></div>
    </div>
  `;
  messagesContainer.appendChild(el);
  el.scrollIntoView({ behavior: 'smooth', block: 'end' });
  return el;
};
```

The three `shimmer-line` variants (`--long`, `--mid`, `--short`) represent approximately 80%, 60%, and 40% of the container width — simulating a three-line text block that the AI is "writing." The shimmer animation (described in Section 04) moves a gradient horizontally across the lines continuously.

When the AI response arrives, `el.remove()` is called and the real message is appended. The transition from shimmer to real content happens in one frame — no visible jump.

The actual AI response time on free-tier models ranged from 1 to 8 seconds in my testing. Without the indicator, users in the 4-8 second range would frequently click "Send" again, thinking their message had not registered. The indicator's sole purpose is to prevent this.

11.7 The `setTimeout` Anti-Pattern I Deliberately Used

In the chat context attachment menu, I needed to close the menu when the user clicks outside it. The standard pattern is adding a click listener to `document` that closes the menu:

```
document.addEventListener('click', closeMenu);
```

But this listener would fire immediately — the same click that opened the menu would also trigger `closeMenu` and immediately close it. The solution was adding the listener asynchronously:

```
const openAttachMenu = () => {
  show(menu);

  setTimeout(() => {
    const closeMenu = (e) => {
      if (!menu.contains(e.target) && e.target !== attachBtn) {
        hide(menu);
        document.removeEventListener('click', closeMenu);
      }
    };
    document.addEventListener('click', closeMenu);
  }, 10);  // Delay past the current click event
};
```

The 10ms timeout defers listener attachment until after the current click event has fully propagated and the browser's event loop moves to the next iteration. The opening click fires, `openAttachMenu` runs, the `setTimeout` schedules the listener addition for "later," and by the time "later" arrives, the opening click's propagation is complete.

This is an acknowledged hack. The technically cleaner solution is `stopPropagation()` on the button's click handler to prevent the event from reaching `document`, combined with a listener added without the timeout. But `stopPropagation()` breaks event bubbling for accessibility tools and any other code that relies on click propagation. The timeout approach is a well-understood workaround that does not break other things.

Section 12 — Notes Subsystem: Full-Text Search & Dynamic SQL Construction

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

12.1 The Notes Feature as the Core of the System

Every design decision in Study-Hub ultimately serves the notes feature. The PDF feature exists because PDFs are research material that enriches notes. The AI chat exists to help users un-

derstand and extend their notes. The authentication system exists because notes are private. Notes are the reason the application exists.

Given that centrality, I treated the notes subsystem as the most important to get right — in both its user experience and its underlying data access patterns.

12.2 The Folder-Note Hierarchy

Notes can be organized into folders, but folders are optional. A note without a folder is "uncategorized." This design choice — optional folders rather than mandatory categorization — came from my own frustration with tools that force you to place everything in a category before you can save it. Sometimes you just want to write a note immediately without stopping to think about where it belongs.

The schema reflects this: `folder_id` is a nullable foreign key:

```
folder_id UUID REFERENCES note_folders(id) ON DELETE SET NULL
```

`ON DELETE SET NULL` means that when a folder is deleted, its notes do not cascade-delete — they become uncategorized instead. This is the correct behaviour for a user-facing system: deleting a folder to reorganize should not silently destroy all notes inside it.

I initially used `ON DELETE CASCADE` (the more common default), discovered during testing that deleting a folder with notes in it deleted the notes silently, and changed to `ON DELETE SET NULL`. This is exactly the kind of schema decision that is painful to change later on a live database with user data, which is why testing schema edge cases early matters.

12.3 The Pinned Notes Ordering Problem

Pinned notes should appear at the top of the list, followed by regular notes in reverse-chronological order. The initial query:

```
SELECT * FROM notes WHERE user_id = $1 ORDER BY updated_at DESC;
```

Did not handle pinning. Adding a `WHERE is_pinned = true` filter was wrong — it would hide unpinned notes entirely.

The correct approach is a composite `ORDER BY`:

```
SELECT * FROM notes WHERE user_id = $1
ORDER BY is_pinned DESC, updated_at DESC;
```

`is_pinned DESC` sorts `true` (1) before `false` (0) in descending order — pinned notes first. `updated_at DESC` then sorts within each group by most recent. The result is: all pinned notes first (most recently updated first among pinned), then all unpinned notes (most recently updated first among unpinned).

This is a SQL pattern I was not initially aware of — ordering by a boolean column as a primary sort key. It is obvious in retrospect but was not my first instinct.

12.4 The Full-Text Search Pipeline

The notes search endpoint is the most complex SQL in the backend, because it combines ownership filtering, archive filtering, folder filtering, pinned filtering, and full-text search — all

optionally, simultaneously.

The dynamic query construction (described in Section 08) produces queries like:

```
SELECT n.*, f.name AS folder_name, f.color AS folder_color
FROM notes n
LEFT JOIN note_folders f ON n.folder_id = f.id
WHERE n.user_id = $1
  AND n.is_archived = false
  AND to_tsvector('english', COALESCE(n.title,'') || ' ' || COALESCE(n.content,''))
      @@ to_tsquery('english', $2)
ORDER BY
  ts_rank(
    to_tsvector('english', COALESCE(n.title,'') || ' ' || COALESCE(n.content,'')),
    to_tsquery('english', $2)
  ) DESC,
  n.is_pinned DESC,
  n.updated_at DESC;
```

One problem I hit: user search input of "machine learning" needed to become `machine & learning` for `to_tsquery`. If the raw string was passed, PostgreSQL would interpret the space as invalid syntax and throw an error. The transformation in the backend:

```
const searchQuery = search
  .trim()
  .split(/\s+/)            // Split on whitespace
  .filter(Boolean)         // Remove empty strings
  .join(' & ');            // Join with AND operator
```

A second problem: single-character search terms like "a" or "I" are English stop words that PostgreSQL removes from the tsvector. Searching for "a" returns zero results because "a" does not exist in any tsvector. I added a minimum search length check (2 characters) to prevent this confusing behaviour.

12.5 The Note Editor: Auto-Save vs. Explicit Save

This was a UX decision with architectural consequences. Auto-save (save every N seconds while editing) versus explicit save (the user clicks a "Save" button) are two legitimate design philosophies.

Auto-save arguments: Users do not lose work. There is no cognitive overhead of remembering to save. Modern cloud documents (Google Docs, Notion) set this expectation.

Explicit save arguments: Auto-save generates many API calls (one every 2-5 seconds while typing). At 300 requests/minute rate limiting, a user who types for 2 minutes generates enough auto-save calls to approach the limit. More importantly, auto-save complicates the experience when you want to abandon an edit — clicking away from a note should discard unsaved changes if the user has not explicitly committed them.

I implemented explicit save with an "unsaved changes" indicator. When the editor content differs from the last saved version, a dot appears on the Save button. Clicking away from a note with unsaved changes shows a browser-native `confirm()` dialog ("You have unsaved changes. Leave anyway?").

The architectural consequence: I had to track "dirty state" — whether the editor's current content differs from the saved version. This required storing the last-saved content separately from the editor's current content and comparing them on every change:

```
let savedContent = '';
let savedTitle = '';

editorContent.addEventListener('input', () => {
  const isDirty = editorContent.value !== savedContent ||
                  editorTitle.value !== savedTitle;
  savBtn.classList.toggle('has-changes', isDirty);
});
```

This is a common pattern in document editors, but implementing it correctly — including resetting `savedContent` after each successful save, and handling the case where the user opens a different note while the editor is dirty — required more state management than I initially anticipated.

12.6 The Tags System: Comma-Separated Input to Array

Tags are stored as a PostgreSQL `TEXT[]` array (Section 08). On the frontend, users enter tags as comma-separated text: `machine learning, ai, study`. The tag input needed to parse this into a clean array before sending to the server.

```
const parseTags = (input) =>
  input
    .split(',')
    .map(t => t.trim().toLowerCase())
    .filter(t => t.length > 0 && t.length <= 50);
```

Lowercasing ensures that "AI" and "ai" are the same tag. Trimming removes accidental whitespace around commas. Filtering removes empty strings (from trailing commas: "machine learning, ai,") and tags that are too long.

On the display side, tags are rendered as pill badges. The colour of each pill is deterministically derived from the tag text using a hash function, so the same tag always gets the same colour across notes without storing a colour preference per tag.

12.7 Soft Delete via Archiving

Notes are never hard-deleted immediately — that would be irrecoverable. Instead, clicking "Archive" sets `is_archived = true`. Archived notes are hidden from the main view but can be retrieved from the archive view. Only "Delete permanently" from the archive view triggers `DELETE FROM notes WHERE id = $1 AND user_id = $2`.

This two-stage deletion is intentional: accidents happen. A user who archives a note by mistake can retrieve it. A user who permanently deletes can not. The cognitive cost — an extra step to permanently delete — is the point.

The architectural implication: the `is_archived` flag must be included in every notes query. The default is always `WHERE is_archived = false`. An easy bug to introduce is a new query that forgets this filter and surfaces archived notes in the main view. I caught this once during testing: a search query I added did not include the archive filter, and archived notes appeared in search results. The fix was adding `AND n.is_archived = false` to the dynamic conditions builder unconditionally, making it impossible to forget.

Section 13 — PDF Subsystem: Upload Pipeline, Text Extraction & Storage

---

13.1 Why PDFs Are Architecturally Different From Notes

Notes are text the user writes directly into the application. They are small, structured, and entirely managed by Study-Hub. PDFs are binary files from the external world — arbitrary sizes, arbitrary internal structure, arbitrary encoding. Handling them required a separate, more complex pipeline that spans the HTTP layer, the filesystem, and a parsing library.

The PDF pipeline involves five distinct steps: receiving the file from the browser, saving it to disk, extracting its text content, storing metadata in the database, and serving the file back to the browser on demand. Each step has its own failure modes, and the failure of any step must be handled without corrupting the others.

13.2 Multer: The Upload Middleware

File uploads do not arrive as JSON. They arrive as `multipart/form-data` — a binary encoding that interleaves file data with form field values in a specific format that `express.json()` cannot parse.

I used Multer to handle this:

```
// middleware/upload.js
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    const userDir = path.join(uploadDir, req.user.id);
    if (!fs.existsSync(userDir)) {
      fs.mkdirSync(userDir, { recursive: true });
    }
    cb(null, userDir);
  },
  filename: (req, file, cb) => {
    const { v4: uuidv4 } = require('uuid');
    const ext = path.extname(file.originalname);
    cb(null, `${uuidv4()}${ext}`);
  }
});

const upload = multer({
  storage,
  limits: { fileSize: (parseInt(process.env.MAX_FILE_SIZE_MB) || 50) * 1024 * 1024 },
  fileFilter: (req, file, cb) => {
    if (file.mimetype === 'application/pdf') {
      cb(null, true);
    } else {
      cb(new Error('Only PDF files are allowed'), false);
```

```
    }
  }
});
```

Three configuration decisions here:

Per-user subdirectories. Each user's PDFs are stored under `uploads/pdfs/{userId}/`. This isolation means a filesystem-level compromise of one user's directory does not expose another's. It also makes it straightforward to delete all of a user's files when their account is deleted — just delete the directory.

UUID filenames. The original filename (`lecture-3.pdf`) is replaced with a UUID (`832e87e7.pdf`). This prevents several problems: filename collisions between users uploading files with the same name, path traversal attacks (a malicious filename like `../../server.js` becomes a harmless UUID), and information leakage (the on-disk name revealing nothing about content or ownership).

File type filtering. The `fileFilter` function checks `file.mimetype`. This is not a security measure — MIME types can be spoofed. It is a usability guard that gives an immediate, clear error if the user accidentally tries to upload a Word document instead of a PDF. Real security against malicious file content would require analysing the file's magic bytes or sandboxed parsing.

13.3 The PDF Parsing Problem

After saving the file to disk, I needed to extract its text content for two purposes: storing it in the database so the AI can read it, and enabling future text-based search features.

I used the `pdf-parse` library:

```
// services/pdfService.js
const pdfParse = require('pdf-parse');
const fs = require('fs');

const extractText = async (filePath) => {
  try {
    const dataBuffer = fs.readFileSync(filePath);
    const data = await pdfParse(dataBuffer);
    return {
      text:      data.text,
      pageCount: data.numpages,
    };
  } catch (err) {
    console.error('PDF parsing failed:', err.message);
    return { text: '', pageCount: 0 };
  }
};
```

The first edge case I hit: `pdf-parse` throws on encrypted PDFs (PDFs with password protection or edit restrictions). The `try/catch` returns empty text rather than crashing — the PDF is still stored, it just has no extracted text. The user can still view it in the browser (the file itself is unencrypted for viewing if view-only protection is used) but cannot use it for AI context.

The second edge case: image-only PDFs — scanned documents where every page is a rasterized image with no text layer. `pdf-parse` returns an empty string for these. True text extraction from image-based PDFs requires Optical Character Recognition (OCR), which is computation-

ally intensive. `tesseract.js` can do this in Node.js but adds significant processing time per page and memory usage. I chose not to implement OCR, documenting it as a known limitation.

The third edge case: very large PDFs. A 500-page academic textbook can have millions of characters of extracted text. Storing this in a PostgreSQL `TEXT` column is fine — `TEXT` is unlimited in PostgreSQL — but passing it all to the AI has hard limits (discussed in Section 14). The extraction stores the full text; the truncation happens at AI call time.

13.4 The Upload Controller: Orchestrating the Pipeline

```
// controllers/pdfController.js
const uploadPdf = async (req, res, next) => {
  const file = req.file;
  if (!file) {
    return res.status(400).json({ success: false, error: 'No file uploaded' });
  }

  try {
    // Step 1: Extract text (non-blocking failure)
    const { text, pageCount } = await pdfService.extractText(file.path);

    // Step 2: Store metadata in database
    const result = await query(
      `INSERT INTO pdfs
         (user_id, folder_id, filename, original_name, file_size, page_count, extracted_text)
       VALUES ($1, $2, $3, $4, $5, $6, $7)
       RETURNING *`,
      [
        req.user.id,
        req.body.folder_id || null,
        file.filename,                    // UUID on-disk name
        req.body.original_name || file.originalname,
        file.size,
        pageCount,
        text,
      ]
    );

    res.status(201).json({ success: true, pdf: result.rows[0] });

  } catch (err) {
    // Clean up the uploaded file if database insertion fails
    if (file && file.path) {
      fs.unlink(file.path, () => {});  // fire-and-forget cleanup
    }
    next(err);
  }
};
```

The cleanup in the `catch` block is a crucial detail: if the database insertion fails after the file is already saved to disk, there would be an orphaned file with no database record. Subsequent requests cannot reference it (no ID in the database), and it wastes disk space indefinitely. The `fs.unlink()` call removes the file on failure. I use a callback rather than `await` because the

cleanup is best-effort — if the unlink also fails, I log it but do not compound the error.

13.5 In-Browser PDF Viewing: The iframe Solution

I wanted users to be able to view PDFs within the Study-Hub interface, without downloading them. The cleanest solution leveraged the browser's built-in PDF renderer:

```
<iframe id="pdf-iframe" src="" title="PDF Viewer" type="application/pdf"></iframe>
```

When the user opens a PDF, JavaScript sets the `src` to the streaming endpoint:

```
const viewPdf = (pdf) => {
  const token = Storage.getToken();
  pdfIframe.src = `/api/pdfs/${pdf.id}/stream?token=${token}`;
  showPdfViewer();
};
```

The streaming endpoint on the server:

```
const streamPdf = async (req, res, next) => {
  // Token comes from query param (iframe cannot send headers)
  const token = req.query.token;

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    const result = await query(
      'SELECT * FROM pdfs WHERE id = $1 AND user_id = $2',
      [req.params.id, decoded.id]
    );

    if (!result.rows.length) {
      return res.status(404).json({ success: false, error: 'PDF not found' });
    }

    const pdf = result.rows[0];
    const filePath = path.join('./uploads/pdfs', pdf.user_id, pdf.filename);

    if (!fs.existsSync(filePath)) {
      return res.status(404).json({ success: false, error: 'File not found on disk' });
    }

    res.setHeader('Content-Type', 'application/pdf');
    res.setHeader('Content-Disposition', `inline; filename="${pdf.original_name}"`);
    fs.createReadStream(filePath).pipe(res);

  } catch (err) {
    return res.status(401).json({ success: false, error: 'Invalid token' });
  }
};
```

`fs.createReadStream(filePath).pipe(res)` streams the file to the browser in chunks rather than loading the entire file into memory first. For a 50MB PDF, loading into memory would require 50MB of RAM per concurrent request. Streaming reads a chunk, sends it, reads the next chunk — memory usage stays constant regardless of file size.

`Content-Disposition: inline` tells the browser to display the file rather than download it. `Content-Disposition: attachment` would trigger a download prompt.

The `fs.existsSync(filePath)` check before streaming is essential for the ephemeral disk situation: after a Render redeploy, the database record exists but the file is gone. Without the check, `fs.createReadStream()` would throw an unhandled exception that crashes the request. With the check, it returns a clean 404 with a message the frontend can handle gracefully.

13.6 The Ephemeral Disk Discovery in Context

I want to describe how I discovered the ephemeral disk problem, because it illustrates the class of issues that only appear in production.

In development, I tested the full upload-store-view-delete cycle many times. It worked flawlessly. I deployed to Render.com. I uploaded several test PDFs. They appeared correctly in the library, displayed in the viewer, and worked as AI context. I then pushed a small CSS fix. Render redeployed automatically (this is the feature I wanted — continuous deployment). I refreshed Study-Hub. The PDF library showed the same records. I clicked one. The viewer showed a blank frame.

I checked the server logs. The streaming endpoint was returning 404: "File not found on disk." The database record was intact — `SELECT * FROM pdfs` returned the row with correct metadata. But the file at `uploads/pdfs/{userId}/{filename}` did not exist. The entire `uploads/` directory was gone.

I had not read Render's free-tier documentation carefully enough. The ephemeral filesystem is mentioned, but I had mentally categorized it as "something that happens to the database" (which Render manages separately and persistently). I had not considered that my application's own disk — where Multer saves uploaded files — was part of the ephemeral container.

The immediate mitigation: extracted text is stored in the database. Even when the PDF file is gone from disk, the AI can still read and discuss its content via the `extracted_text` column. The PDF viewer and download features are broken for files uploaded before the last redeploy, but the knowledge extracted from the PDF persists.

The long-term solution I documented: migrate file storage to Cloudflare R2 (free tier with 10GB storage), which stores files in object storage that survives container restarts entirely. This requires replacing the Multer disk storage with an S3-compatible upload to R2. The database and serving logic stay the same; only the storage destination changes.

Section 14 — AI Chat: Context Injection, Model Fallback & Auto-Note Creation

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

14.1 The Core Ambition and Its Constraint

The AI chat feature was the reason I built Study-Hub. Every other feature exists to feed it. The ambition was simple: a conversational AI that knows what you are studying. Ask it ques-

tions about a PDF you uploaded. Ask it to summarize your notes on a topic. Have it create a structured note from a conversation.

The constraint was equally simple: I could not afford the infrastructure to do this properly. The architecturally correct approach — Retrieval-Augmented Generation (RAG) — requires a vector database to store document embeddings and a semantic search system to retrieve the most relevant chunks at query time. Every production RAG system I evaluated (Pinecone, Weaviate, Chroma with cloud hosting) had pricing that exceeded my zero-cost requirement beyond trivial usage.

So I designed an approximation. This section documents both what I built and where it falls short of the architecturally ideal solution.

14.2 Why RAG Would Have Been Better

RAG works by splitting documents into chunks (typically 256-512 tokens each), computing a vector embedding for each chunk (a numerical representation of semantic meaning), storing those embeddings in a vector database, and at query time, computing the embedding of the user's question and finding the chunks with the highest cosine similarity — the chunks most semantically relevant to the question.

This has two major advantages over my approach:

Precision. Only the relevant portions of a document are included in the AI's context. A 500-page textbook is not sent as 15,000 characters of arbitrary beginning text — instead, the specific passages that answer the user's question are retrieved.

Scale. A RAG system can handle arbitrarily large document collections. The user can have hundreds of PDFs; only the relevant chunks from relevant documents are selected per query. My approach — manually truncating and prepending full document text — cannot scale beyond a handful of documents.

I want to be explicit about this gap. The AI feature in Study-Hub works well for its current scale (small collections, specific documents explicitly selected by the user). For larger-scale personal knowledge management, it is architecturally insufficient.

14.3 The Context Injection Strategy

My approximation of RAG is direct context injection: I take the text of explicitly attached notes and PDFs, truncate it to fit within the model's context window, and prepend it to the system prompt.

```
// llmService.js - context string construction
let contextStr = '';

if (context.pdfs.length === 1) {
  const p = context.pdfs[0];
  contextStr += `\n\n--- ATTACHED PDF: "${p.original_name}" (${p.page_count} pages) ---\n`;
  contextStr += p.extracted_text
    ? p.extracted_text.slice(0, 15000)
    : '[No text extracted from this PDF]';
  contextStr += '\n--- END OF PDF ---\n';

} else if (context.pdfs.length > 1) {
```

```
  // Multiple PDFs: reduced budget per document
  contextStr += "\n\n--- USER'S ATTACHED PDFs ---\n";
  context.pdfs.forEach(p => {
    contextStr += `\n**${p.original_name}** (${p.page_count} pages):\n`;
    contextStr += p.extracted_text
      ? p.extracted_text.slice(0, 5000) + '\n---\n'
      : '[No text]\n---\n';
  });
}
```

The truncation limits (15,000 characters for a single PDF, 5,000 per PDF when multiple) were chosen empirically. Free-tier models on OpenRouter typically have a context window of 8,192 to 32,768 tokens. 15,000 characters is approximately 4,000 tokens — leaving space for the system prompt, conversation history, and the user's message. These numbers are not theoretically optimal; they are practical guesses validated against the models I was using.

The hard cutoff (`.slice(0, 15000)`) is the crudest possible truncation. It cuts mid-sentence, mid-paragraph, or even mid-word. A better approach would truncate at a paragraph boundary, or summarize the truncated portion and append the summary. I did not implement this. For documents where the most important content is at the beginning (many academic papers have their key findings in the abstract and introduction), the truncation works adequately. For documents where the relevant content is at the end, it fails.

14.4 The OpenRouter Integration and the Model Fallback Chain

OpenRouter's free tier is real but rate-limited. Any single model might be rate-limited at peak times, returning HTTP 429 errors. A single-model integration would fail ungracefully during these periods.

I designed a fallback chain:

```
const MODELS = [
  'openrouter/free',              // Auto-routed to whatever is available
  'meta-llama/llama-3.3-70b-instruct:free',  // Llama 3.3 70B
  'google/gemma-3-12b-it:free',  // Gemma 3 12B
];

const chat = async (history, context, user) => {
  let lastError;

  for (const model of MODELS) {
    try {
      console.log(`[LLM] Trying model: ${model}`);
      const text = await callOpenRouter(apiKey, model, messages);
      console.log(`[LLM] Success with: ${model}`);
      return text;
    } catch (err) {
      console.warn(`[LLM] Failed with ${model}: ${err.message}`);
      lastError = err;
      // Continue to next model
    }
  }
```

```
  throw new Error(`All models failed. Last error: ${lastError?.message}`);
};
```

`openrouter/free` is tried first — it auto-routes to whatever capable model is available right now, which maximizes success probability. If it fails (rate limit, model overloaded), we try Llama explicitly. If that fails, we try Gemma. All three are free tier.

In practice, `openrouter/free` succeeded on the vast majority of requests. The fallback chain was needed occasionally during peak hours but the logging confirmed it was invoked and succeeded without user-visible errors in those cases.

14.5 The System Prompt and the `[[CREATE_NOTE]]` Protocol

The system prompt is the most important engineering artifact in the AI feature. It defines the AI's identity, capabilities, and a custom protocol:

```
You are Study-Hub AI, a smart study assistant integrated into a personal knowledge management system.

You have access to the user's notes and uploaded PDF documents. You can:
1. Answer questions based on stored notes and PDFs
2. Summarize notes or PDF content
3. Generate structured study notes from PDFs
4. Help organize and understand study materials
5. Answer general knowledge questions

When the user asks you to CREATE or SAVE notes, generate the note content between special tags:
[[CREATE_NOTE]]
# Note Title

Note content in markdown format...
[[/CREATE_NOTE]]

The system will automatically detect this and save the note to their library.

Always be concise, helpful, and academic in tone. Format responses with markdown when appropriate.
```

The `[[CREATE_NOTE]]` protocol is the most interesting piece of prompt engineering in the project. When a user says "create a note summarizing this PDF," the AI includes its generated note between the special tags. The backend detects these tags, extracts the content, saves it to the database, and strips the tags from the displayed response:

```
// chatController.js
if (aiResponse.includes('[[CREATE_NOTE]]')) {
  const noteMatch = aiResponse.match(/\[\[CREATE_NOTE\]\]([\s\S]*?)\[\[\/CREATE_NOTE\]\]/);
  if (noteMatch) {
    const noteContent = noteMatch[1].trim();
    const titleMatch  = noteContent.match(/^#\s+(.+)/m);
    const noteTitle   = titleMatch ? titleMatch[1] : 'AI Generated Note';

    const noteResult = await query(
      'INSERT INTO notes (user_id, title, content, color) VALUES ($1,$2,$3,$4) RETURNING *',
      [req.user.id, noteTitle, noteContent, '#1a2035']
    );
    savedNote = noteResult.rows[0];
```

```
    // Strip tags from displayed response
    aiResponse = aiResponse
      .replace(/\[\[CREATE_NOTE\]\][\s\S]*?\[\[\/CREATE_NOTE\]\]/, '')
      .trim();
    aiResponse += '\n\n  **Note saved to your Notes library!**';
  }
}
```

The regex `\[\[CREATE_NOTE\]\]([\s\S]*?)\[\[\/CREATE_NOTE\]\]` captures everything between the tags. The `[\s\S]*?` pattern matches any character including newlines (`.` in JavaScript does not match newlines by default), and `*?` is non-greedy (stops at the first `[[/CREATE_NOTE]]` rather than the last).

One failure mode I encountered in testing: the AI occasionally placed text after the closing tag but before its final response. The `replace()` and `.trim()` removed the tags but left the trailing content in the wrong position. I added a more robust clean-up that specifically removes the tag block and appends the success message:

```
aiResponse = aiResponse
  .replace(/\[\[CREATE_NOTE\]\][\s\S]*?\[\[\/CREATE_NOTE\]\]/g, '')  // remove block
  .replace(/\n{3,}/g, '\n\n')   // collapse excessive newlines left behind
  .trim();
```

14.6 Context Window Management: The Real Bottleneck

The context window of a language model is the maximum amount of text (measured in tokens) it can process at once. Free models on OpenRouter have context windows ranging from 8,192 to 32,768 tokens. A token is roughly 0.75 words in English.

The context for a Study-Hub chat message includes: - System prompt: ~250 tokens - Injected PDF content: up to ~4,000 tokens (15,000 chars) - Injected note content: up to ~1,000 tokens - Conversation history (20 messages): up to ~5,000 tokens - User's message: ~50-200 tokens

Total: approximately 10,000-11,000 tokens. This fits within most free-tier context windows but is close to the limit of smaller models. I hit context window errors occasionally during development when both a large PDF and a long conversation were combined.

My mitigation was to limit conversation history to the last 20 messages rather than the full history:

```
SELECT role, content FROM chat_messages
WHERE session_id = $1
ORDER BY created_at ASC
LIMIT 20;
```

For very long conversations, this means the AI loses memory of messages beyond the most recent 20. This is a known limitation — the AI might say "as I mentioned earlier" referring to something it can no longer see. The alternative (summarizing older messages into a compressed summary) would require an additional AI call per message, doubling the API usage and latency.

14.7 Streaming Bugs: The Async Response Problem

The initial version of the chat feature had a streaming bug that took me an embarrassingly long time to diagnose.

I had implemented the frontend message submission as:

```
const response = await API.chat.sendMessage(sessionId, body);
appendMessage({ role: 'assistant', content: response.assistantMessage.content });
```

This worked correctly in isolation. The bug appeared when users sent messages quickly: if a user sent a second message before the first AI response arrived, both responses would render in the wrong order — the response to message 2 appeared before the response to message 1.

The root cause: `async/await` does not queue multiple calls. If `sendMessage` is called twice before either resolves, both are in-flight simultaneously. Their responses race to resolve. The second response might arrive first (if the first AI call happened to hit a slower model in the fallback chain).

The fix was the `isStreaming` flag:

```
let state = { isStreaming: false };

const sendMessage = async () => {
  if (state.isStreaming) return;  // Block concurrent sends

  state.isStreaming = true;
  sendBtn.disabled = true;

  try {
    const response = await API.chat.sendMessage(...);
    appendMessage(response.assistantMessage);
  } finally {
    state.isStreaming = false;  // Always re-enable, even on error
    sendBtn.disabled = false;
  }
};
```

The `finally` block is essential: without it, a failed AI call would leave `isStreaming = true` permanently, making the chat input non-functional for the rest of the session. `finally` runs regardless of whether the `try` block succeeded or threw — it is the correct place for cleanup code that must always run.

14.8 The Auto-Title Mechanism

New chat sessions are created with the title "New Chat." After the first message is sent, the session is automatically renamed to the first ~60 characters of the user's message:

```
if (session.rows[0].title === 'New Chat') {
  const autoTitle = content.trim().slice(0, 60) +
                    (content.length > 60 ? '...' : '');
  await query(
    'UPDATE chat_sessions SET title = $1 WHERE id = $2',
    [autoTitle, sessionId]
  );
}
```

This happens server-side, after saving the user's message but before calling the AI. If I generated the title from the AI's response instead, there would be a period where the session shows "New Chat" while the AI is thinking — the title would flicker from "New Chat" to something meaningful only after the response arrived. Using the user's first message as the title updates the session list immediately, giving the user instant visual confirmation that their conversation is named.

The `slice(0, 60)` truncation preserves the beginning of the message. For questions like "Can you explain the photoelectric effect and how it relates to quantum mechanics?", the title becomes "Can you explain the photoelectric effect and how i..." — not perfect but immediately meaningful and recognizable.

Section 15 — Holistic Architecture: Data Flow & Cross-Cutting Decisions

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

15.1 Zooming Out: The System as a Whole

The preceding sections each examined one subsystem in isolation. This section steps back to look at Study-Hub as a unified system: how the parts interact, what cross-cutting properties were designed in, and what architectural tensions existed and how they were resolved.

15.2 The Complete Request Lifecycle

The most revealing way to understand a system is to trace one request through it completely. I will trace the most complex request: a user sends a chat message with a PDF attached.

Step 1 — User Interaction (Browser) The user has opened a chat session, attached "Lecture 3.pdf" via the attachment menu, typed "Summarize the key arguments," and pressed Enter.

```
chat.js: input keydown event fires with key === 'Enter'
→ state.isStreaming check: false, proceed
→ state.isStreaming = true, sendBtn.disabled = true
→ Optimistically append user message to DOM
→ Show typing indicator (shimmer skeleton)
→ API.chat.sendMessage(sessionId, { content, context_pdf_ids: ['pdf-uuid'] })
```

Step 2 — HTTP Request (Network)

```
api.js: request('POST', '/api/chat/sessions/{id}/messages', body)
→ Storage.getToken() → reads JWT from localStorage
→ Headers: { Content-Type: application/json, Authorization: Bearer eyJ... }
→ Body: JSON.stringify({ content: 'Summarize...', context_pdf_ids: ['pdf-uuid'] })
→ fetch() → TCP connection to Render server → HTTP/1.1 POST
```

Step 3 — Express Middleware Pipeline (Server)

```
Helmet: adds Content-Security-Policy, X-Frame-Options headers to response queue
CORS: validates Origin header, adds Access-Control-Allow-Origin
```

```
Rate limiter: increments counter for this IP, max 300/min - still within limit
JSON parser: req.body = { content: '...', context_pdf_ids: [...] }
Morgan: logs "POST /api/chat/sessions/... - pending"
Router: matches POST /chat/sessions/:sessionId/messages
Auth middleware:
  → reads 'Bearer eyJ...' from Authorization header
  → jwt.verify(token, JWT_SECRET) → decoded = { id: 'user-uuid', exp: ... }
  → SELECT id, name FROM users WHERE id = $1 → finds user
  → req.user = { id: 'user-uuid', name: 'Arnab', ... }
  → next() → controller runs
```

Step 4 — Controller Logic (chatController.sendMessage)

```
→ Parse and normalise context IDs
→ Verify session ownership: SELECT * FROM chat_sessions WHERE id=$1 AND user_id=$2
→ Save user message: INSERT INTO chat_messages (role='user', content='...') RETURNING *
→ buildContext():
    SELECT extracted_text, original_name, page_count FROM pdfs
    WHERE user_id=$1 AND id IN ($2)
    → returns PDF row with extracted_text (first 15,000 chars sent to AI)
→ Fetch history: SELECT role, content FROM chat_messages
    WHERE session_id=$1 ORDER BY created_at ASC LIMIT 20
```

Step 5 — LLM Service Call (External Network)

```
llmService.chat(history, context, user):
→ Build contextStr: prepend PDF text to system prompt
→ Build messages array: [system+context, ...history, user_message]
→ fetch('https://openrouter.ai/api/v1/chat/completions', {
    method: 'POST',
    headers: { Authorization: 'Bearer {OPENROUTER_KEY}', 'HTTP-Referer': '...' },
    body: JSON.stringify({ model: 'openrouter/free', messages, max_tokens: 2048 })
  })
→ OpenRouter routes to available free model
→ AI generates response (1-8 seconds)
→ response.json() → data.choices[0].message.content
→ Return AI response text
```

Step 6 — Post-Processing

```
→ Check aiResponse for '[[CREATE_NOTE]]' → not present in this case
→ INSERT INTO chat_messages (role='assistant', content=aiResponse) RETURNING *
→ Check session title: currently 'New Chat' is false (second message in session)
   (if it were the first: UPDATE chat_sessions SET title=$1 WHERE id=$2)
→ res.json({ success: true, userMessage: {...}, assistantMessage: {...}, savedNote: null })
```

Step 7 — Frontend Receives Response

```
→ fetch() resolves with data
→ api.js returns data
→ chat.js: removeTypingIndicator(typingEl)
→ appendMessage({ role: 'assistant', content: aiResponse })
→ buildMessageBubble(): renders AI response as Markdown HTML
→ scrollToBottom()
→ state.isStreaming = false, sendBtn.disabled = false
→ input.focus()
```

Total elapsed time from key press to rendered response: approximately 2-9 seconds, domi-

nated by the OpenRouter API call.

15.3 Cross-Cutting Concern: Error Boundaries

Every async operation in Study-Hub can fail. The design principle I applied was: failure at any level should be localized — it should not cascade into adjacent features or unrecoverable states.

At the server level: Express's error handler middleware catches any unhandled exceptions from route handlers that call `next(err)`. The server does not crash; it returns a 500 response and continues running.

At the AI call level: If all three models in the fallback chain fail, the error is caught in `chatController.js`. The AI response is replaced with an error message string, which is then saved to the database and returned as a normal assistant message. The user sees the error in the chat bubble — they can re-try without losing the conversation.

At the frontend level: Every `API.*` call is wrapped in `try/catch`. Failures show a toast notification and reset the UI state. The `finally` block in `sendMessage` ensures `isStreaming` is always reset, even on errors.

At the PDF streaming level: The `fs.existsSync()` check before streaming means a missing file returns a clean 404 rather than an unhandled exception.

15.4 Cross-Cutting Concern: The Ownership Model

Every database query that touches user data includes a `user_id` constraint. This is enforced not by the architecture (there is no framework-level enforcement) but by discipline — a code rule I applied to every query I wrote.

The consequence is that data isolation is robust: a user cannot access another user's data regardless of what ID they provide in the request URL. The JWT identifies them; the query checks their identity; the resource is only returned if both match.

This property is not formally verified — there is no automated test that checks every query for the `user_id` constraint. A future improvement would be a database-level row security policy (PostgreSQL Row Level Security) that enforces ownership at the database level, making it impossible for an incorrectly written query to return unauthorized data.

15.5 The Module Communication Architecture

The four frontend modules (Auth, Notes, PDFs, Chat) are deliberately isolated. They do not import each other. They do not have direct references to each other's functions.

Cross-module communication uses browser custom events on `window`:

```
// Convention: event names use 'source:action' format
'chat:attach-pdf'  → Chat module attaches a PDF to context
'chat:attach-note' → Chat module attaches a note to context
'nav:switch'       → App module switches the active view
'notes:created'    → Notes module announces a new note was created
```

`app.js` is the only module that has explicit references to all other modules (to call their `init()` and `refresh()` methods). Between modules, all communication is indirect through events.

This architecture has a concrete benefit: I can completely rewrite the Chat module without touching Notes or PDFs, as long as the custom events contract is preserved. The modules are coupled to the event protocol, not to each other's implementations.

15.6 Performance Characteristics Under Free-Tier Constraints

The free Render tier introduces performance characteristics that a paid deployment would not have.

Cold starts. The service sleeps after 15 minutes of inactivity. The first request after sleep takes 30-60 seconds to respond — Render provisions a fresh container, loads Node.js, loads application dependencies, establishes a database connection, and then processes the request. I mitigated this with a health-check endpoint (`/api/health`) that can be pinged regularly to keep the service warm.

Database query latency. The managed PostgreSQL database on Render's free tier is in the same data centre as the web service, so database round-trips are fast (1-5ms). On paid tiers with geographic distribution, query latency could increase if the web service and database are in different regions — but on free tier, both run in the same region automatically.

AI call latency. OpenRouter's free models are the dominant source of latency: 1-8 seconds per request depending on current load, model complexity, and response length. This is irreducible given the free-tier model constraint. The typing indicator (Section 11) manages this latency from a UX perspective.

Concurrent user limitations. Render's free tier provides a single container with limited CPU and memory. Under high concurrent load, requests queue and latency increases. The rate limiter (300 req/min) is both a security measure and a practical upper bound on concurrent activity that the free tier can handle without degradation.

15.7 Decisions Made for Simplicity That I Would Revisit

Looking at the system as a whole, several decisions that made sense for initial development would need to be revisited for a production system with real users at scale:

No request queuing. AI calls from multiple concurrent users block in parallel. Under high load, all available connections in the pool could be waiting for OpenRouter responses, starving other requests. A job queue (Bull, BullMQ) would decouple AI calls from the HTTP request lifecycle.

No caching layer. The same note list is re-fetched from the database on every view switch. Redis caching with appropriate invalidation would reduce database load. Not critical at current scale, but necessary at 10x usage.

No search indexing. Full-text search is computed at query time. A GIN index on the tsvector was added, but for very large note collections (thousands of notes), a dedicated search service would be more performant.

Single-container deployment. The Node.js server handles HTTP serving, business logic, AI calls, and file serving all in one process. Horizontal scaling (running multiple instances behind a load balancer) would require session-less architecture (JWT already provides this) and stateless

file serving (requires moving to object storage, which is the right move for other reasons as well).

None of these are failings of the current design — they are appropriate deferred optimizations for a system that does not yet have the usage to justify them.

Section 16 — Deployment: Free-Tier Infrastructure & the Ephemeral Disk

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

16.1 The Gap Between Local Development and Production

Study-Hub worked perfectly on my local machine. Every feature was tested. Notes, PDFs, chat — all functioning. The moment of deployment revealed assumptions I had not known I was making.

This section documents the deployment process, the production configuration differences, the bugs that only appeared in production, and the ongoing operational reality of running a free-tier application.

16.2 Infrastructure as Code: render.yaml

The deployment configuration is committed to the repository as `render.yaml`:

```yaml
services:
  - type: web
    name: study-hub
    runtime: node
    buildCommand: npm install
    startCommand: node backend/server.js
    envVars:
      - key: NODE_ENV
        value: production
      - key: DATABASE_URL
        fromDatabase:
          name: study-hub-db
          property: connectionString
      - key: JWT_SECRET
        generateValue: true
      - key: PORT
        value: 10000

databases:
  - name: study-hub-db
    plan: free
```

The `fromDatabase` directive is Render's mechanism for automatically injecting the database connection string. I do not type the connection string anywhere — Render constructs it from

the provisioned database and injects it as an environment variable. This means the connection string is never in any file that could be committed or shared.

`generateValue: true` for `JWT_SECRET` instructs Render to generate a cryptographically random string and inject it. I never see the value; it exists only in Render's secret store and in the running server's environment. This is the most secure possible handling of a symmetric signing key.

The advantage of `render.yaml` over a manual UI setup: the deployment configuration is version-controlled alongside the code. When I set up a new Render service (e.g., for a staging environment), I click "New Blueprint," point at the repository, and Render reads `render.yaml` — no manual configuration required. Reproducible, documented, auditable.

16.3 What Changes in Production

Several differences between the local development environment and Render's production environment required explicit handling:

PORT. In development, the server runs on `3000`. On Render, the port is assigned by the platform (typically `10000`). The `process.env.PORT || 3000` pattern in `server.js` handles both: in development, the environment variable is unset, so `3000` is used; in production, Render sets `PORT=10000` and the server binds to that.

SSL for the database. Render's managed PostgreSQL requires SSL connections. Locally, I connect to a PostgreSQL instance without SSL. The `ssl` configuration in `database.js`:

```
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production'
    ? { rejectUnauthorized: false }
    : false,
});
```

`rejectUnauthorized: false` was the point of friction. Render uses SSL but with an internal certificate that is not signed by a public CA. Without `rejectUnauthorized: false`, Node.js's TLS implementation rejects the certificate and the connection fails. The "correct" solution is to download Render's root certificate and add it to the trust store — I chose the simpler option with the security tradeoff documented.

Logging format. Morgan in development mode logs colourized, concise output useful for watching in a terminal. In production, Morgan's `'combined'` format generates the standard Apache log format, which is what log aggregation tools expect. The `NODE_ENV` check in `server.js` selects the appropriate format.

File caching. In development, `maxAge: 0` for static files means the browser always re-fetches CSS and JavaScript — essential for seeing changes immediately. In production, `maxAge: '1d'` allows browsers to cache static files for 24 hours, reducing server load and improving perceived performance for returning users.

16.4 The Deployment Pipeline

After the initial setup, every deployment is triggered by a `git push`:

```
git add .
git commit -m "Fix: improve chat error handling"
git push origin main
```

Render detects the push via a GitHub webhook, clones the repository, runs `npm install` (only if `package.json` changed; otherwise it uses cached `node_modules`), starts `node backend/server.js`, waits for the process to respond to the health check at `/api/health`, and then routes traffic to the new instance while stopping the old one.

This is zero-downtime deployment in principle. In practice, on the free tier, there is sometimes a brief period where the new container is starting up while the old one has already stopped — a 2-5 second window where requests might fail with a 503. Acceptable for a personal productivity tool; not acceptable for a production commercial service.

16.5 The Ephemeral Disk: Discovery, Diagnosis, and Response

I described the discovery of the ephemeral disk problem in Section 13. Here I want to document the diagnostic process and my response, because the path from "PDFs are missing" to "I understand why" is instructive.

Symptom: After pushing a CSS fix, existing PDF records showed in the library (database intact) but clicking them showed a blank iframe (file not found on disk).

Initial hypothesis: Bug in the streaming endpoint introduced by my changes. I reviewed the streaming code — no changes had been made to it in that commit.

Second hypothesis: A path configuration issue. I checked `process.env.UPLOAD_DIR`, the file path construction, and the `fs.existsSync()` check. All looked correct.

Diagnosis trigger: I added a debug log to the streaming endpoint:

```
console.log('Looking for file at:', filePath);
console.log('File exists:', fs.existsSync(filePath));
console.log('Upload dir contents:', fs.readdirSync('./uploads').catch(() => 'DIR NOT FOUND'));
```

The log showed: `DIR NOT FOUND`. The `uploads/` directory itself did not exist. This ruled out all code-level explanations — the directory structure was completely absent, not just a missing file.

Root cause: Render's free-tier web services run on ephemeral containers. Every deployment provisions a fresh container. The fresh container has the application code (from Git) but none of the runtime-generated directories or files. The `uploads/` directory created at runtime does not exist in Git (correctly, it is in `.gitignore`). It is recreated on the first file upload after each deployment, but any files uploaded to a previous container are permanently gone.

Response:

1. **Immediate:** I confirmed that `extracted_text` was stored in the database for all affected PDFs. The AI feature continued working — the knowledge extracted from the PDFs persisted even though the original files were gone. The PDF viewer and download features were broken for those files; I added a frontend check that shows "File unavailable (server restarted)" rather than a blank iframe when the streaming endpoint returns 404.
2. **Documentation:** I added a prominent notice to the application's README and to the deployment section of this paper. The limitation is real and users deserve to know about it.

3. **Migration path:** The correct long-term solution is Cloudflare R2 (or Amazon S3), which provides S3-compatible object storage with a genuinely free tier (10GB storage, unlimited egress at origin). The migration requires:
   - Replace Multer's disk storage with an S3-compatible upload (using the `@aws-sdk/client-s3` package or the `@cloudflare/workers-sdk` R2 client)
   - Replace the `fs.createReadStream().pipe(res)` with a redirect to a pre-signed R2 URL or a proxied stream from R2
   - Remove the `uploads/` directory and all `fs.existsSync()` disk checks

The database schema and application logic otherwise stay the same. This is a contained change with high impact — I flagged it as the highest-priority infrastructure upgrade.

16.6 The 90-Day Database Expiry

Render's free PostgreSQL databases are automatically deleted after 90 days. This is disclosed in Render's documentation but easy to miss.

At day 85 of the initial deployment, Render sent an email warning. I had two options: upgrade to a paid tier ($7/month) or export the data and recreate the database. For a personal project, I chose to export:

```
pg_dump --no-acl --no-owner $DATABASE_URL > backup.sql
```

After recreating the free database (which resets the 90-day clock), I imported:

```
psql $NEW_DATABASE_URL < backup.sql
```

This works but it is operational overhead. The long-term resolution is to use a PostgreSQL service with no expiry on the free tier — Supabase (500MB free) and Neon (10GB free) are both viable alternatives with persistent free-tier databases.

16.7 The Cold Start User Experience

The 30-60 second cold start on Render's free tier is the most visible user-facing limitation. A user who returns after a period of inactivity clicks the page, sees nothing happen, waits, waits more, then eventually the application responds.

I addressed this in two ways:

Keep-alive pinging. I configured an external monitoring service (UptimeRobot's free tier) to ping `/api/health` every 5 minutes. This prevents the service from sleeping during normal use. Between sessions (overnight, weekends), it still sleeps — the ping interval is a tradeoff between keeping the service responsive and respecting Render's free-tier intent.

Loading state UI. The application shows a loading spinner while waiting for the initial API call to return. If the first call takes 45 seconds (cold start), the spinner appears for 45 seconds rather than a blank screen. This is better UX — the user knows something is happening — but it does not solve the underlying problem.

16.8 Environment Variables Managed Separately from Code

Secrets added through Render's dashboard (not in `render.yaml`):

Key	Reason for Exclusion From render.yaml
`OPENROUTER_API_KEY`	Secret key — must never be committed
`ALLOWED_ORIGINS`	Deployment-specific, changes per environment
`MAX_FILE_SIZE_MB`	Operational parameter, adjusted without code changes

The principle: anything that should never be publicly visible goes through the dashboard. Anything that is safe to be public (configuration values with no security implications) can go in `render.yaml`. The `JWT_SECRET` uses `generateValue: true` — Render generates and stores it without it ever appearing in a file I control.

16.9 What Production Deployment Taught Me

The gap between "works on my machine" and "works in production" is where the most important learning happens. Study-Hub's deployment revealed:

- Infrastructure assumptions that were invisible in development (ephemeral disk)
- Configuration differences that required explicit handling (SSL, PORT, logging)
- Security considerations that had to be implemented correctly from the start (CORS, environment variables)
- Operational realities that required ongoing attention (database expiry, cold starts)

Every one of these was discoverable in advance by reading the platform documentation more carefully. I did not read carefully enough. That is the real lesson: deployment platform documentation deserves the same attention as programming language documentation.

Section 17 — Lessons Learned, Limitations & Future Work

Study-Hub: Design and Implementation of an AI-Augmented Personal Knowledge Management System
Arnab Mukhar Biswas

17.1 Overview

This section does not summarize what has already been described. It synthesizes the experience of building Study-Hub from start to deployment into three categories: things I would do the same, things I would do differently, and things I would add if the constraints changed. Where relevant, I connect lessons back to their concrete manifestations in the codebase.

17.2 What I Would Do the Same

PostgreSQL over MongoDB. Every time I needed to query across relations — folder note counts, owned resource filtering, joined results — SQL was the right tool. The decision to model data relationally was validated repeatedly. I never wished for a document store.

Vanilla JavaScript over a framework. For the scope of this application, the absence of a build tool was a genuine advantage. The development loop was instantaneous. Debugging required only the browser's developer tools, no source maps. If I were building this for a team

of five people over two years, React would be the right choice. For one person over six weeks, vanilla JavaScript was correct.

IIFE module pattern. Clear public interfaces, private state, explicit dependency ordering — the pattern produced code that remained readable throughout the build. The load order discipline it imposed (no circular dependencies, explicit sequencing) was healthy.

JWT for authentication. The stateless architecture simplified the backend considerably. No session store, no cache dependency, no state synchronization between application instances. For a personal productivity tool with a single backend service, the inability to revoke tokens before expiry is an acceptable tradeoff.

The `[[CREATE_NOTE]]` protocol. This was the most creative piece of engineering in the project. Prompt engineering as an API — defining a text protocol that the AI adheres to, then parsing that protocol server-side — is a genuinely useful pattern for AI-integrated applications. It produces a seamless user experience: "Create a note about photosynthesis" works without any special UI element.

Separation of concerns in the folder structure. The `backend/`, `frontend/`, `scripts/` split with its sub-layering (config, controllers, middleware, routes, services) produced a codebase where every file had a clear home. I never spent time deciding where new code should go — the structure made the decision for me.

17.3 What I Would Do Differently

Read the deployment platform documentation before building. The ephemeral disk problem cost me a debugging session and introduced a user-visible data loss issue. Render's ephemeral filesystem is documented. I did not read that section carefully. Before any deployment, the correct process is to read the platform's "Limitations" and "Free Tier" sections in full and verify that the application's assumptions about storage, memory, and networking are compatible.

Implement token refresh from the start. The JWT expiry UX — users silently losing their session overnight — was a known problem I deferred. The deferred solution (a toast notification before reload) is better than nothing but worse than proper token refresh. This should have been part of the initial authentication design, not a patch after deployment.

Add indexes before discovery. I added the GIN index on notes full-text search vectors after observing latency on a realistic dataset. Database indexes should be designed alongside the schema, not discovered through performance problems. The rule of thumb: any column that appears in a `WHERE` clause or `ORDER BY` on a large table should have an index.

Implement more robust Markdown parsing or use a lightweight library. My custom Markdown renderer (Section 04) has edge cases — nested formatting, escaped characters, lists within blockquotes — that a complete parser handles correctly. The `marked` library (3KB gzipped) would have been a reasonable dependency that I rejected out of an excess of independence. For a feature this central to the AI chat experience, library quality would have been worth the dependency.

Design the CORS configuration with the actual deployed URL in mind. The CORS bug (Section 07) cost debugging time that could have been saved by a five-minute check of what Render's URL format produces. Configuration bugs are rarely interesting to debug. They are prevented by being methodical about configuration before deployment.

Consider the PDF streaming auth mechanism earlier. The URL-token approach for the PDF iframe (Section 10) is a known security tradeoff — tokens in URLs appear in logs. I accepted it because the alternative (pre-signed URLs with short-lived tokens) was complex. But

it should have been a deliberate design decision made early, not discovered as a constraint mid-implementation.

17.4 Financial Constraint: What It Cost and What It Forced

Operating entirely on free infrastructure was the most creatively constrained I have been in any engineering project. It forced decisions I would not have made with unlimited budget — and some of those decisions turned out to be architecturally interesting.

What the constraint cost:

The most significant technical cost was the inability to implement RAG. The gap between Study-Hub's context injection approach and a proper RAG system is not marginal — it is the difference between finding the most relevant 500 tokens from a document collection and blindly including the first 15,000 characters of whatever the user explicitly attached. For large document collections, the current approach fails. For small, carefully curated collections, it works adequately.

The second cost was AI quality. Llama 3.3 70B and Gemma 3 12B are capable models, but they produce lower-quality responses than GPT-4 or Claude Opus, especially on nuanced academic content. The free models occasionally lose context, produce repetitive text, and misunderstand complex questions. For a study tool that is supposed to help comprehension, this quality gap matters.

The third cost was operational complexity: 90-day database expiry, ephemeral disk management, cold start mitigation, keep-alive pinging — these are maintenance tasks that a paid service would eliminate.

What the constraint forced:

The context injection strategy — crude but zero-cost — produced a working AI feature under severe constraints. Designing around limits produces creative solutions. The fallback model chain is a resilience pattern that I would use even with budget — the ability to degrade gracefully to a weaker model rather than failing completely is valuable regardless of cost.

The constraint also enforced rigorous prioritization. I could not add "just one more feature" because every feature had to be justified against the infrastructure limits. The result is a focused application that does three things well rather than eight things adequately.

17.5 Known Limitations

Image-based PDFs produce no extracted text. Scanned documents where pages are rasterized images are invisible to the AI. OCR is the solution; it was not implemented due to computational cost.

Full-text search is English-only. The `to_tsvector('english', ...)` call uses the English language dictionary for stemming and stop-word removal. Notes in other languages will produce degraded search results. A proper multi-language implementation would detect the note's language and use the appropriate tsvector configuration.

No real-time collaboration. Study-Hub is single-user. There is no mechanism for sharing notes or collaborating on documents. This was a deliberate scope decision, not an oversight.

JWT tokens cannot be invalidated. Password changes do not invalidate existing sessions. An attacker who captures a valid token can use it until expiry (24 hours). Token blacklisting requires infrastructure (Redis or a database table) that was not implemented.

AI has no true long-term memory. The conversation history limit of 20 messages means AI memory is session-bounded and recency-bounded. The AI cannot recall a conversation from last week. True persistent memory would require either summarization (additional AI calls) or embedding-based retrieval (vector database).

PDF files are lost on redeployment. On Render's free tier, the ephemeral disk means all uploaded PDF files are deleted on every deployment. Extracted text persists in the database. The original files do not.

17.6 Future Work: Priority Order

The following improvements are listed in order of priority, where priority is defined as the combination of user impact and implementation feasibility:

1. Cloudflare R2 for PDF storage (High impact, Moderate effort) Replace Multer's disk storage with R2 object storage. This eliminates the ephemeral disk problem permanently. The R2 free tier (10GB, $0.015/GB beyond that) is sufficient for personal use. The migration is contained: modify `upload.js` and `pdfController.js`, leave everything else unchanged.

2. JWT token refresh (High impact, Moderate effort) Implement a refresh token mechanism using httpOnly cookies for the refresh token and short-lived JWTs for access. This eliminates the "session expired" disruption for users who leave the application open.

3. GIN search indexes (Medium impact, Low effort) The GIN index on `to_tsvector(title || content)` should be part of the schema by default. This is a one-line addition to `schema.sql` that significantly improves search performance at scale.

4. Markdown parser upgrade (Medium impact, Low effort) Replace the custom Markdown renderer in `helpers.js` with `marked` or `micromark`. These are small, well-tested, CommonMark-compliant parsers that handle edge cases my implementation does not.

5. Token blacklist for password changes (Medium impact, Moderate effort) A `token_blacklist` table in PostgreSQL that records invalidated token JTIs (JWT IDs). The auth middleware checks the blacklist on every request. Adds one indexed database query per authenticated request.

6. RAG architecture (High impact, High effort, Requires budget) The correct long-term solution to AI context. Requires: a vector database (Supabase's built-in pgvector extension is free within the database quota), an embedding model (OpenAI's `text-embedding-3-small` at $0.00002/1K tokens, or a free embedding model via HuggingFace), and a chunking strategy for documents. This is a significant architectural addition that would transform the AI feature from "adequate for small collections" to "works for large knowledge bases."

7. Supabase migration (Medium impact, Moderate effort) Moving from Render's expiring free PostgreSQL to Supabase's persistent free-tier PostgreSQL (500MB, no expiry) eliminates the 90-day database rotation overhead. Supabase also provides pgvector, which would enable item 6.

17.7 Closing Reflection

Study-Hub is a working, deployed application. It has real functionality — notes, PDFs, and AI chat — on real infrastructure, serving real requests. It was built under severe financial constraints that forced every architectural decision to be deliberate and justified.

The constraint of free infrastructure is commonly treated as a hobbling limitation. I found it clarifying. Every tool had to earn its place. Every feature had to justify its complexity. The result is a codebase that is lean, comprehensible, and relatively free of speculative architecture.

The gap between what I built and what I would build with resources is real — primarily the RAG system and paid AI models. But the gap between what Study-Hub does today and what its users need is small. It solves the problem I set out to solve: a unified environment where a student can write notes, upload documents, and ask an AI questions about both, without leaving the application.

That the entire system runs on the free tier of a hosting provider is, from one perspective, a limitation. From another, it is a proof of concept: capable, production-grade AI-augmented applications can be built and deployed at zero recurring cost, if you are willing to design around the constraints rather than pretend they do not exist.

That is the central finding of this paper.